Kru — Project Plan & Milestones

Nov 1, 2026 · @Amin Haiqal

Overview

Kru is a headless, local-first engine that runs AI agent CLIs (Claude Code, Codex, Gemini, Cursor and others) as persistent teammates. Every agent gets a durable mailbox, a workspace and a thread that survives provider switches and crashes. Work is delivered one turn at a time, and agents message each other through the same mailbox.

Pitch: *Give every agent a mailbox. Kru delivers the work and never loses a message.*

The name: *kru* is Malay for "crew" — a crew of agents that hand work to each other. One syllable, easy to say, easy to type as a command.

Who uses it

- Developers who want several agent CLIs to work on one codebase, each with its own workspace and memory.
- Teams that put agents behind their own tools: a Slack bot, a CI job, an internal dashboard.
- Builders who need an engine they can embed, not a desktop app.

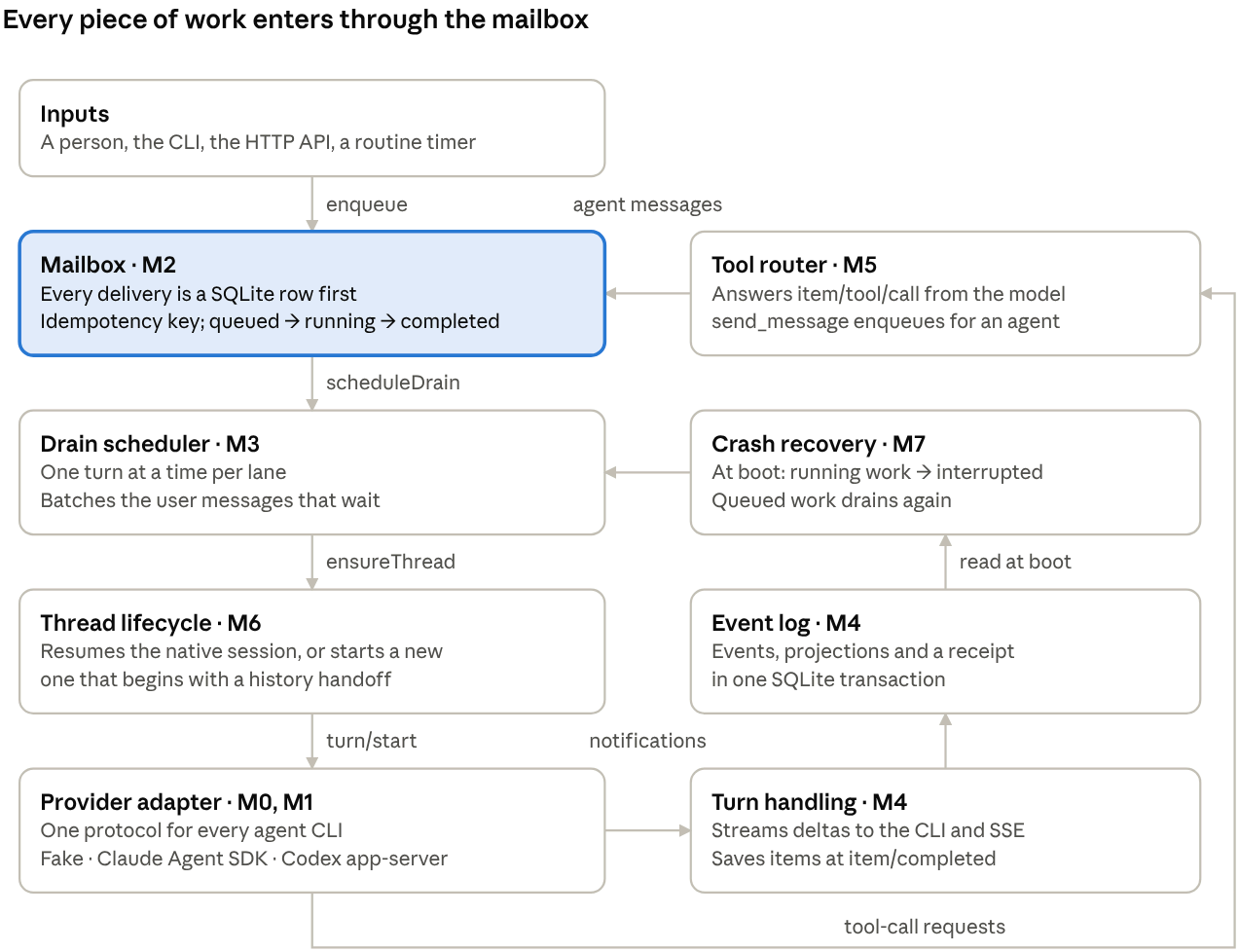
How it differs from OpenBot

	OpenBot	Kru
Form	Electron desktop app	Daemon, CLI, HTTP + SSE API, TypeScript library
Audience	End users	Developers and integrators
Scope	Browser, mobile, voice, team servers	The core engine only
License	PolyForm Noncommercial 1.0.0	MIT (allows commercial use)
Code	—	Written from scratch from the ideas; no copied code

The license row is why the last row matters: OpenBot's code cannot go into an MIT project, so study a file, close it, then write your own version.

Architecture

Work flows down the left column and results flow back up the right. Agent messages loop back into the same mailbox, so there is only one execution path.



Kru message lifecycle · 9 parts, labelled by milestone

Core concepts

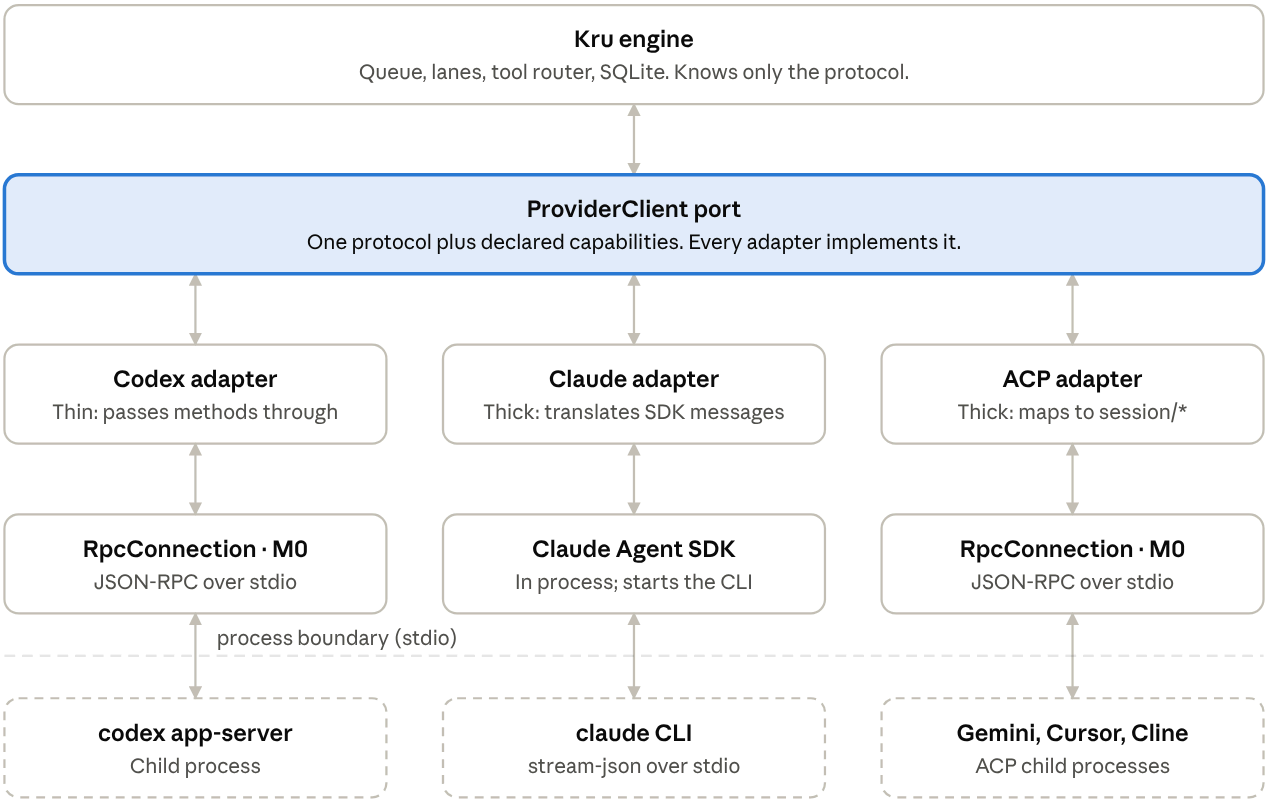
- **Delivery:** one unit of work for one agent, stored before anything runs.
- **Lane:** a queue that runs one turn at a time — an agent's main chat, or one of its side threads.
- **Thread vs provider session:** the thread is Kru's and permanent; the session is the CLI's and disposable.
- **Handoff:** a size-limited summary of recent history, prepended to the first turn of a new session.

- **Turn:** one `turn/start` to `turn/completed` exchange with a provider.

Provider layers: ports and adapters

Kru's engine depends on one interface, never on a specific agent. This is hexagonal architecture, also called ports and adapters: the `ProviderClient` interface is the port, and each agent CLI plugs in through its own adapter. Adding an agent means adding one adapter; the engine does not change.

The engine talks to one port; adapters absorb each agent difference



Kru provider layers · engine, port, 3 adapters, transports, agent processes

Arrows run both ways: commands go down, events and tool-call requests come up. Dashed boxes are separate processes that Kru starts and can lose at any time.

What each layer owns

Layer	Owns	Knows about	Milestone
Engine	What runs and when: queue, lanes, tool router, SQLite	The protocol only	M2–M7

Layer	Owns	Knows about	Milestone
`ProviderClient` port	The contract: protocol methods and declared capabilities	Nothing below it	M1
Adapters	Translation between the protocol and one agent's native API	One agent	M1
Transport	Framed messages over stdio, request ids, timeouts	Bytes and JSON-RPC	M0
Agent process	The model, its tools, its own session files	Nothing about Kru	External

The shared language. OpenBot uses the Codex App Server protocol as the language every adapter speaks, because Codex already had a clean JSON-RPC design. Kru keeps the same method names (see Core types) but gives each method's params and result a TypeScript type, so the compiler checks every adapter. JSON-RPC framing stays inside the transport.

How each adapter translates

Kru protocol	Codex adapter	Claude adapter	ACP adapter
`thread/start`	Sent as-is	Creates an SDK session itself	`session/new`
`turn/start`	Sent as-is	Calls the SDK's `query()` with the prompt	`session/prompt`
`item/agentMessage/delta` (up)	Received as-is	Built from SDK assistant messages	Built from `session/update` (`agent_message_chunk`)
`turn/completed` (up)	Received as-is	Built when the SDK result message arrives	Built when `session/prompt` returns

Kru protocol	Codex adapter	Claude adapter	ACP adapter
`item/tool/call` (up, request)	Received as-is	Built from the SDK tool callback	Built from ACP tool requests

The Codex adapter is thin. The Claude and ACP adapters are thick — about 2,400 and 3,150 lines in OpenBot — because they act as a JSON-RPC server: they answer requests themselves and build notifications from what their agent emits.

One turn through the layers (Claude)

1. The engine calls `turn/start` with the thread id and the prompt.
2. The Claude adapter calls `query()` and answers at once with the new turn id.
3. The SDK writes to the `claude` CLI's stdin and reads its stdout as stream-json.
4. Text arrives; the adapter emits `item/agentMessage/delta` notifications up to the engine.
5. The model calls `send_message`; the adapter sends an `item/tool/call` request up and waits for the engine's answer.
6. The SDK reports the result; the adapter emits `turn/completed`.

The engine sees the same six events it would see from Codex.

Where state lives

Layer	Keeps	When it is lost
Transport	Pending requests by id	Rejected on process exit (M0)
Adapter	Native session ids, the active turn	Session resumed or replaced with a handoff (M6)
Engine	Threads, deliveries, messages in SQLite	Never: SQLite is the source of truth

Rules that keep the layers clean

1. Dependencies point inward: the engine imports the port, never an adapter. Adapters are chosen in one registry.
2. An adapter never writes to SQLite or to the screen. It only emits protocol events.

3. The transport knows nothing about turns, threads or tools.
4. Each adapter declares a `capabilities` object (compaction mode, steering, subagents). The engine uses a feature when it is present, instead of reducing every agent to the lowest common denominator.
5. Every adapter passes the same conformance test suite: the fake, Codex and Claude run the same file.

Why it pays off

- **One file per agent.** A new provider is a new adapter; nothing else changes.
- **Free tests.** The fake agent is just another adapter, so M2–M7 run with no model and no API key.
- **Contained failures.** A crashed CLI becomes one `exit` event; the engine marks the turn interrupted and nothing above needs to know why.
- **Provider switching.** The thread never belonged to Codex or Claude, so it can move between them (M6).
- **Comparable results.** Every agent looks the same to the engine, so the same task can run on each and be measured — the base for evals and cross-provider review after v0.1.

Stack, ground rules and layout

Use TypeScript on Bun with `bun:sqlite`, the same stack as OpenBot, so you can compare files side by side. Use plain `async` / `await`; learn Effect later as a separate project.

Ground rules

1. Build the fake provider first. Every test runs against it: deterministic, free, and able to stall or crash on command. OpenBot does the same in `scripts/provider-fakes/`.
2. Write your version before you read theirs. Then read their file and note what they handled that you missed.
3. Every milestone ends with a test that proves its "Done when" line, and a small user-facing result.
4. SQLite is the source of truth. Provider session files are cache.
5. Never retry a turn whose outcome is unknown. Mark it `interrupted` and let a person decide.

Repository layout

```
kru/
  src/
    rpc/          JSON-RPC over stdio (M0)
    providers/    ProviderClient + adapters: fake, claude, codex (M1)
    store/        SQLite, events, projections, migrations (M2, M4)
    mailbox/      enqueue, delivery state machine (M2)
    drain/        lanes and scheduler (M3)
    turns/        notification handling, delta buffer (M4)
    tools/        tool router, send_message, history tools (M5, M6)
    threads/      provider sessions, handoff (M6)
    boot/         crash recovery (M7)
    api/          HTTP + SSE server (v1.0)
    cli/          kru command (every milestone)
  test/
    fakes/        scripted fake agents: echo, stall, crash, tool-caller
  examples/
  docs/
```

Core types

Five types carry the whole engine. Start with these and change them as the milestones teach you more.

```
// Who sent a delivery. Users, agents and routines share one queue.
type Sender =
  | { kind: "user" }
  | { kind: "agent"; agentId: string }
  | { kind: "routine"; routineId: string };

type DeliveryStatus = "queued" | "starting" | "running" | "completed" |
"failed" | "interrupted" | "cancelled";

// One unit of work for one agent. A row in SQLite, never only in memory.
interface Delivery {
  id: string;
  recipientAgentId: string;
  laneId: string | null;          // null = the agent's main chat
  sender: Sender;
  text: string;
  replyToMessageId: string | null;
  expectsReply: boolean;
```

```
  status: DeliveryStatus;
  idempotencyKey: string | null;  // UNIQUE: a retry returns the first
receipt
  createdAt: string;
}

// Kru's thread. Survives provider switches.
interface Thread { id: string; agentId: string; laneId: string | null }

// The provider's private session, bound to one thread. Disposable.
interface ProviderSession { threadId: string; provider: string;
externalSessionId: string; retiredAt: string | null }

// Every provider adapter implements this. One protocol for all of them.
interface ProviderClient {
  readonly provider: string;
  start(): Promise<void>;
  stop(): Promise<void>;
  request<T>(method: string, params: unknown, timeoutMs?: number):
Promise<T>;
  respond(id: string | number, result: unknown): void;
  respondError(id: string | number, error: { code: number; message: string
}): void;
  on(event: "notification", fn: (n: { method: string; params: any }) =>
void): void;
  on(event: "request", fn: (r: { id: string | number; method: string; params:
any }) => void): void;
  on(event: "exit", fn: (error: Error) => void): void;
}
```

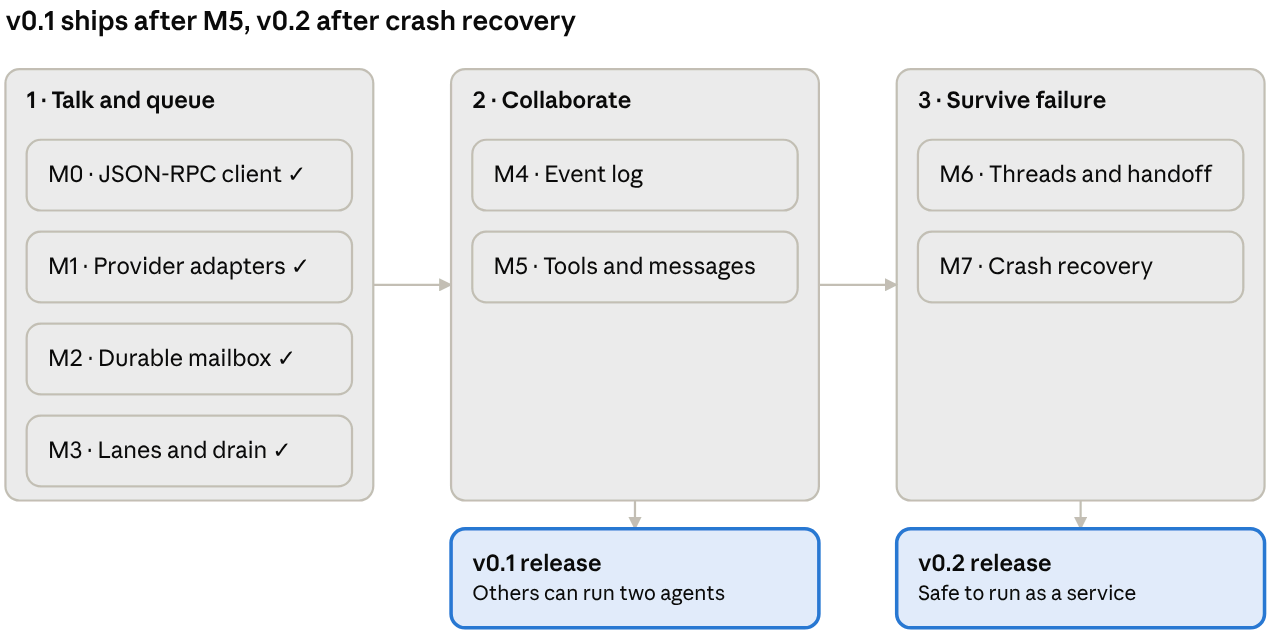
The protocol methods every adapter must speak:

Direction	Method	Meaning
Kru → provider	`thread/start`, `thread/resume`	Open or reopen a native session
Kru → provider	`turn/start`, `turn/interrupt`	Run one turn, or stop it
Provider → Kru	`turn/started`, `turn/completed`	Turn boundaries, with status

Direction	Method	Meaning
Provider → Kru	`item/started`, `item/agentMessage/delta`, `item/completed`	Streamed output and tool steps
Provider → Kru	`thread/tokenUsage/updated`	Context fill, for compaction
Provider → Kru (request)	`item/tool/call`	The model calls a Kru tool and waits for the answer

Milestones

Eight milestones, each isolating one idea. After M5 Kru is usable by other people (v0.1). After M7 it is crash-safe (v0.2). Each milestone lists what to build, which OpenBot files to read after you build it, the test that proves it, and what a user can do with it.



Kru roadmap · 8 milestones in 3 phases, 2 releases

Each phase depends on the one before it; inside a phase, build in order.

M0 — JSON-RPC over stdio (the transport) · Done

Status: done. Released as v0.0.1 on 2026-11-02, with the review fixes in v0.0.2 the same day. The spec below is what was built; the implementation record at the end of this milestone says how.

Purpose. Kru does not run a model. It starts agent CLIs as child processes and talks to them over stdin and stdout. M0 builds the bottom layer of the hexagon: a generic connection that sends and receives JSON-RPC messages reliably. It knows nothing about Codex, Claude, turns or threads, and it is proven against a fake agent. In M1 the Codex and ACP adapters use it directly; the Claude adapter uses the Agent SDK instead.

Public API

```
interface RpcConnectionOptions {
  command: string;
  args: string[];
  env?: Record<string, string>;
  requestTimeoutMs: number;
  maxLineBytes: number;        // one message = one line; longer = fatal
  maxQueuedWriteBytes: number; // stdin backpressure cap
  maxBadLines: number;         // consecutive unparsable lines before failing
}

class RpcConnection {
  constructor(options: RpcConnectionOptions);
  start(): void;
  stop(): Promise<void>;       // ends the whole process tree
  request<T>(method: string, params: unknown,
    options?: { timeoutMs?: number; signal?: AbortSignal; decode?: (value:
unknown) => T }): Promise<T>;
  notify(method: string, params?: unknown): Promise<void>;
  // The transport sends the answer: the resolved value, or an error if it
throws.
  onRequest(handler: (request: { id: string | number; method: string; params:
unknown; signal: AbortSignal }) => Promise<unknown>): void;
  on(event: "notification", listener: (n: { method: string; params: unknown
}) => void): void;
  on(event: "diagnostic", listener: (d: Diagnostic) => void): void; // stderr
| bad-line | late-response | write-blocked
  on(event: "exit", listener: (error: ProcessExitError) => void): void; //
fires exactly once
}
```

Design rules

1. **Pure transport.** No method names, turns or threads inside this module. Agent-specific behaviour, such as a liveness probe that needs a cheap method, belongs to the M1 adapter.
2. **Frame on bytes.** Split on the newline byte before decoding UTF-8, so a character split across two chunks is never corrupted.
3. **Every request gets exactly one answer.** The handler's value becomes the result; a thrown error becomes `-32603`; no handler becomes `-32601`. A deadlock from a missing listener is impossible.
4. **A bad line is a diagnostic, not a crash.** Messages before and after it in the same chunk are still handled. After `maxBadLines` consecutive bad lines the connection fails, so a truly broken CLI is still restarted.
5. **Backpressure.** When `stdin.write()` returns false, wait for `drain`. Queued bytes above `maxQueuedWriteBytes` reject the write with a clear error instead of growing memory.
6. **Late answers are visible.** Remember recent timed-out ids in a bounded set. An answer that arrives later becomes a `late-response` diagnostic with the method and the delay.
7. **Stale-process guard.** Events from an old child are ignored after a restart, and request ids never reset, so an old answer cannot resolve a new request.
8. **Exit reports the real reason.** On exit, wait briefly for stderr to end, then reject every pending request with a `ProcessExitError` that carries the last stderr line.
9. **Stop ends the whole tree.** Terminate the child and its children; force-kill after a grace period, so no MCP servers are left behind.
10. **One line has a size limit.** Past `maxLineBytes` the stream cannot recover, so the connection fails. OpenBot sets 256 MiB, because one Codex thread resume reached 21 MB.

Fake agent (`test/fakes/fake-agent.ts`) — a script that speaks the protocol and, on command: answers normally, answers out of order, stalls, crashes, sends notifications, sends a request back, writes split, merged or invalid lines, writes to stderr, and stops reading stdin. Every later milestone reuses it.

Tests — `bun test`, no network, no API key, about 2 seconds.

#	Test	Proves
1	`ping` returns `pong`	Basic round trip
2	100 concurrent requests answered in random order	Each answer reaches its own caller
3	Error response	Rejects with the code and message
4	Fake stalls	Rejects with a timeout error
5	Five notifications	Arrive in order
6	Fake sends a request; the handler answers	The fake receives the answer and echoes it back
7	One message split across two chunks; two messages in one chunk	Framing is correct
8	A bad line between two good lines in one chunk	Both good messages handled; one `bad-line` diagnostic; connection stays up
9	Fake exits during a request	Every pending request rejects; `exit` fires once, with the last stderr line
10	`stop()`, then `request()`	Child tree is gone; the new request rejects at once
11	Fake writes to stderr	Reported as a diagnostic, never parsed as protocol
12	Fake stops reading; 50 MB of writes	Memory stays bounded; writes wait or fail cleanly
13	Timeout, then the fake answers late	One `late-response` diagnostic; no crash, no double resolve
14	Fake sends a request with no handler set	It receives a `-32601` error

After every test: the pending map is empty, no child process is running, and there are no unhandled promise rejections.

Done when: all 14 tests and the hygiene checks pass, every time.

You can now:

```
$ kru rpc ping --fake
pong (1.4 ms)

$ kru rpc ping --fake --stall
error: request "ping" timed out after 2000 ms
```

Stretch: send `initialize` to a real `codex app-server --listen stdio://` and print the answer.

Read after: `src/backend/app-server-client.ts`, `src/backend/jsonl.ts`, `src/backend/protocol.ts`.

Out of scope: providers, SQLite, queues, retries.

Improvements over OpenBot

OpenBot's transport is production-hardened. Kru keeps its byte framing, line limit, stale-process guard, stderr-before-exit and process-tree kill, and changes six things:

Area	OpenBot	Kru M0
Separation	The Codex client mixes transport with Codex logic (turn interrupts, resume params, history paging); ACP and Pi have their own transports	One generic transport for every JSON-RPC agent; agent logic lives in adapters
Bad line	Any unparsable line ends the process and every agent on it; valid messages earlier in the chunk are dropped	Diagnostic and skip; fail only after `maxBadLines` in a row
Backpressure	`stdin.write()` result ignored; writes queue without limit	Wait for `drain`; cap queued bytes
Late answers	Ignored silently after a timeout	Reported as `late-response`, so the engine can reconcile a turn that did start
Unanswered requests	The tool router answers unknown methods, but the transport does not guarantee an answer	The transport always answers exactly once

Area	OpenBot	Kru M0
Types	`params: unknown`, checked deep in the engine	Decoded at the boundary through `decode`

One trade-off: OpenBot's fail-fast on a bad line is a deliberate safety choice. Skipping is safe here only because each NDJSON line is independent, which is why the `maxBadLines` limit stays.

Implementation record

The M0 transport meets every design rule above and all six improvements over OpenBot. Code: aminhaiqal/kru; exports in `src/rpc/index.ts`, tests in `test/rpc.test.ts`.

Shipped in v0.0.1

- Generic JSON-RPC connection: byte framing, strict UTF-8, CRLF accepted, incoming and outgoing line limits, concurrent requests, notifications, inbound request handlers.
- Timeouts, cancellation and result decoding; bounded stdin writes with `drain`; a `late-response` diagnostic from a bounded set of the last 1,024 timed-out ids.
- Process-tree cleanup: each child in its own process group on POSIX, SIGTERM then SIGKILL; `taskkill /T /F` on Windows.
- The deterministic fake agent, `kru rpc ping --fake` with stall mode, and `kru --version`.
- 32 tests and GitHub Actions CI.

Beyond the plan

- An oversized outgoing frame rejects only its own write, not the connection.
- Cancelling or timing out a queued request removes its unsent frame.
- Handler replies from a previous process are suppressed after a restart.
- Each spawned process owns its own listeners, maps, buffers and handler controllers.
- Read-only `pid`, `pendingRequestCount` and `queuedWriteBytes` for observability.

Review fixes (v0.0.2)

Found in review	Fix
Codex omits the `jsonrpc` field; a strict check would turn every Codex message into a bad line	Accept a missing field, reject a wrong version, and set outgoing frames with the `jsonrpcField` option (default `true`); a Codex connection factory passes `false`
A 100 ms SIGTERM grace can corrupt a real agent's session files	Configurable shutdown grace, default 3 s; configurable stderr wait, default 500 ms; tests keep shorter values
Children in their own process group no longer receive Ctrl+C	Every registered connection stops on SIGINT or SIGTERM; handlers stay until cleanup ends; the CLI returns conventional signal exit codes
A hung shutdown makes the user wait out the grace period	A second signal sends SIGKILL to every tracked process group at once, skips the remaining waits, reaps the children and exits
After `kill -9` of Kru, children run on as orphans	Each child's PID and process group are saved with its start identity, for M7 recovery, and removed after cleanup
A crash during a write could leave a half-written ownership record	Records are published with an fsynced temporary file and an atomic rename; the reader ignores unpublished, truncated and invalid records
macOS can refuse a process-group signal during reaping	The transient permission error is handled

Verification

- Strict TypeScript checking passes.
- All 32 v0.0.1 tests pass in about 1.2 s with Bun 1.4.4 on macOS; ten repeated runs pass all 320 cases with the per-test hygiene checks.
- Ten repeated suite runs in Linux CI, with manual workflow dispatch.
- An exact-byte `initialize` smoke test passes against real Codex 0.165.0, locally and in CI.
- v0.0.2 tests cover envelope compatibility, graceful stop, the second-signal escalation, process ownership, the ownership reader and the signal lifecycle.

Open items

- [x] ~~Tag v0.0.2 before M1 starts~~
- [x] ~~A second Ctrl+C force-kills every tracked process group and exits~~
- [x] ~~Write the ownership file atomically; ignore truncated and invalid records~~
- [x] ~~Real Codex `initialize` smoke test with `jsonrpcField: false`~~
- [ ] Confirm the `late-response` diagnostic names the method as well as `delayMs`
- [ ] Validate process-tree cleanup on Windows

M1 — One protocol, many providers (the port and adapters) · Done

Status: done on 2026-11-03. Fake, Codex and Claude adapters run behind one typed port and pass the same conformance suite. Five follow-ups and the v0.0.3 tag are listed at the end of the implementation record, below the improvements table.

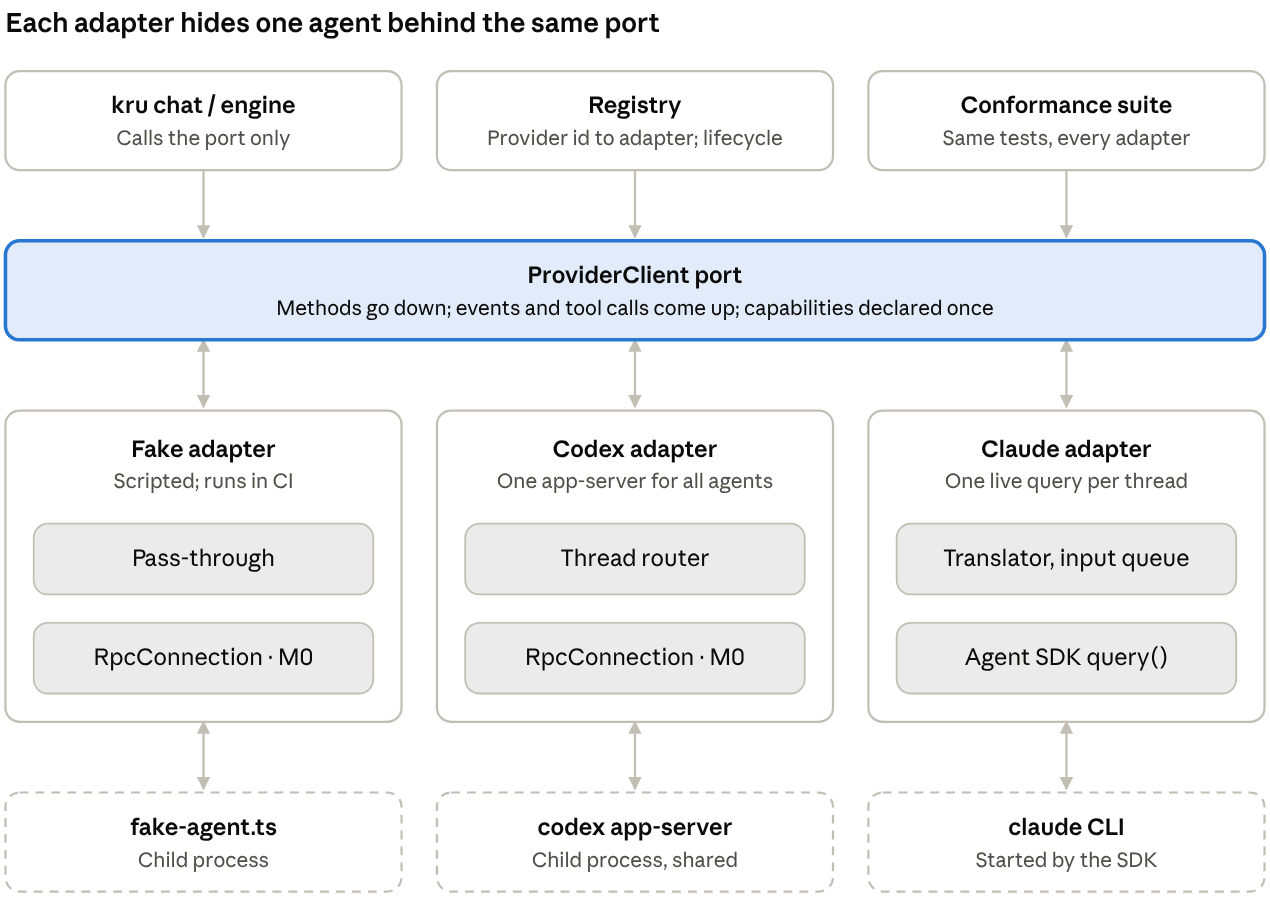
Purpose. M1 builds the `ProviderClient` port and the adapters behind it. After M1 the engine runs a turn on the fake, Codex or Claude without knowing which. M1 is also where token use becomes visible: every adapter must report what each turn cost, because the main reason people leave OpenBot is how many tokens it uses.

Build order

1. Typed protocol: one TypeScript map of methods (params and result) and one of events. The method names stay those in Core types.
2. A `capabilities` object that every adapter declares.
3. The `ProviderClient` port and a registry that picks the adapter.
4. The fake adapter on the M0 fake agent, with scripted token counts.
5. The conformance suite, written against the fake first.
6. The Codex adapter on `RpcConnection` through the existing Codex factory (`jsonrpcField: false`). One app-server process serves every Codex agent as separate threads.
7. The Claude adapter around the Agent SDK. It is the thick one, so it comes last.
8. The `kru chat` demo, which prints the usage of each turn.

Architecture

M1 has three parts: the port (the contract), the adapters (translation), and the registry with the conformance suite (wiring and proof).



M1 architecture · port, registry, conformance suite, 3 adapters, 3 processes

Commands go down; events and tool calls come up. Fake and Codex reuse the M0 transport; Claude goes through the Agent SDK.

The port carries four kinds of traffic

- **Down, methods:** `thread/start`, `thread/resume`, `turn/start`, `turn/interrupt`.
- **Up, events:** `turn/started`, message deltas, `item/completed`, usage, `turn/completed`.
- **Up, one request:** a tool call, which the engine's handler must answer.
- **Declared once:** `capabilities`.

Inside an adapter

Every adapter has the same four jobs:

1. Translate methods down into the agent's native calls.
2. Translate native output up into protocol events.

3. Bridge tool calls: the native tool call goes to `onToolCall(call, signal)`, and the result goes back in native form.
4. Normalise usage and errors.

An adapter keeps two pieces of state only: Kru's thread id mapped to the native session, and the active turn of each thread. A turn moves from `starting` to `running` to `completed`; a process exit from any state ends it `interrupted`.

How the three adapters differ

	Fake	Codex	Claude
Transport	M0 `RpcConnection`	M0 `RpcConnection`, `jsonrpcField: false`	Agent SDK in process; it starts the `claude` CLI
Processes	One per test	One app-server for every Codex agent	One live `query()` per active thread, closed when idle
Start / resume	Passed through	`thread/start` with instructions and tools / `thread/resume`	`query({ sessionId })` / `query({ resume })`
One turn	`turn/start`	`turn/start`	A message pushed into the thread's input queue; the turn ends at the SDK result message
Text streaming	Passed through	`item/agentMessage/delta`	SDK stream events (`includePartialMessages: true`)
Kru tools	Scripted	Declared at thread start; Codex sends `item/tool/call`	An in-process MCP server (`createSdkMcpServer`); the model sees `mcp__kru__send_message`
Usage source	Scripted	`thread/tokenUsage/updated`, last-turn value	SDK result message: input, cache read, cache write, output
Size	Trivial	Thin	Thick

Claude: streaming input

The Claude adapter does not start a new `query()` for each turn. It starts one per thread whose prompt is an async queue, and each `turn/start` pushes a message into it. The session stays alive between turns: no process restart, and the prompt cache stays warm. OpenBot does the same with `AsyncMessageQueue` in `src/backend/claude-client.ts`.

The tool-call path

1. The model calls a Kru tool; the adapter receives the native call and calls `onToolCall(call, signal)`.
2. The engine's handler does the work — in M5, it enqueues a delivery — and returns a result.
3. The adapter sends the result back in native form, and the model continues.

As in M0, every tool call gets exactly one answer, and `signal` aborts when the turn is interrupted or the process exits.

Registry and lifecycle

- `registry.create(provider, options)` returns a `ProviderClient`. Nothing else in Kru constructs an adapter.
- Adapters start on first use. The registry hands out one shared Codex instance.
- On a signal, the registry stops every adapter through the M0 signal handling.

The conformance suite (below) runs against `registry.create("fake" | "codex" | "claude")`. It is the executable definition of the port: an adapter that passes it works with the engine.

Folder layout

```
src/providers/
  protocol.ts          typed method and event maps, capabilities (depends on
nothing)
  port.ts              ProviderClient interface
  registry.ts          provider id → adapter factory, lifecycle
  fake/adapter.ts
  codex/adapter.ts     uses src/rpc (M0)
  claude/adapter.ts    uses @anthropic-ai/claude-agent-sdk
  claude/input-queue.ts
test/providers/conformance.test.ts
```

Dependencies point inward: adapters import `protocol.ts` and their own transport; the engine imports `port.ts` and `registry.ts` only.

Port API

```
interface ProtocolMethods {
  "thread/start":   { params: { cwd: string; instructions: string; tools:
ToolDefinition[]; model: string;
                      promptProfile: "minimal" | "coding" | "full";
                      permissions: { sandbox: "read-only" | "workspace-
```

```
write"; approvalPolicy: "on-request" | "never" } }; result: { threadId:
string } };
  "thread/resume":  { params: { threadId: string }; result: { threadId:
string } };
  "turn/start":     { params: { threadId: string; input: string; model:
string }; result: { turnId: string } };
  "turn/interrupt": { params: { threadId: string; turnId: string }; result:
Record<string, never> };
}

interface ProtocolEvents {
  "turn/started":              { threadId: string; turnId: string };
  "item/agentMessage/delta":   { threadId: string; turnId: string; itemId:
string; delta: string };
  "item/completed":            { threadId: string; turnId: string; item:
ThreadItem };
  "thread/tokenUsage/updated": { threadId: string; turnId: string;
inputTokens: number; cachedInputTokens: number;
                                outputTokens: number; contextWindow: number
| null; estimated: boolean };
  "turn/completed":            { threadId: string; turnId: string; status:
"completed" | "failed" | "interrupted"; error?: string };
}

interface ProviderCapabilities {
  compaction: "events" | "request" | "native" | "handoff" | "none";
  steer: boolean;               // can add input to a running turn
  multiThreadProcess: boolean;  // one process serves many threads
  usage: "exact" | "estimated";
  promptCache: boolean;
}

interface ProviderClient {
  readonly provider: string;
  readonly capabilities: ProviderCapabilities;
  start(): Promise<void>;
  stop(): Promise<void>;
  request<M extends keyof ProtocolMethods>(method: M, params:
ProtocolMethods[M]["params"]): Promise<ProtocolMethods[M]["result"]>;
  on<E extends keyof ProtocolEvents>(event: E, listener: (params:
ProtocolEvents[E]) => void): void;
```

```
  onToolCall(handler: (call: ToolCall, signal: AbortSignal) =>
Promise<ToolResult>): void;
  onExit(listener: (error: Error) => void): void;
}
```

Design rules

1. The engine imports the port and the registry only, never an adapter.
2. An adapter never writes to SQLite or the screen. It only emits protocol events.
3. A turn that ends normally ends with exactly one `turn/completed`. If the process dies, the adapter emits `exit` and does not invent a completion: it does not know how the turn ended. The waiting turn rejects with an unknown outcome, and the engine decides what it was (M7).
4. Every turn reports usage. When the provider gives counts, `estimated` is false; when it does not, the adapter estimates (about 4 bytes a token) and sets `estimated: true`. A turn with no usage event fails conformance.
5. A stable prefix. Instructions and tool definitions are fixed when the thread starts and never injected per turn, so the provider's prompt cache stays valid. Changing them starts a new session, and the reason is logged.
6. A small surface. Kru's own instructions stay under 8 KB (about 2,000 tokens) and an agent sees at most 6 tools in M1. A test fails when either grows, as OpenBot's `check:agent-context` does, but with a lower limit.
7. Shared processes. When `multiThreadProcess` is true (Codex), one process serves all agents of that provider. Claude threads with no turn for 10 minutes are closed and resumed on demand.
8. Capabilities, not the lowest common denominator. The engine uses a feature when an adapter declares it.
9. A prompt profile per agent. `minimal` is a short custom prefix with no built-in tools. `coding` is a short coding prefix with read tools for read-only agents or coding tools for writable ones. `full` keeps the agent CLI's own system prompt, project settings and standard tools, and is the baseline for measuring the saving (a follow-up). For Claude, `minimal` and `coding` use a custom `systemPrompt`, `settingSources: []` and an explicit tool list.

Conformance suite — one test file; every adapter runs it.

#	Test	Proves
1	Start a thread, run a turn	Deltas arrive, then one `turn/completed`
2	Interrupt during a turn	Status `interrupted`; still exactly one `turn/completed`
3	The model calls an `echo` tool	The handler's result reaches the model
4	The process exits during a turn	`turn/completed` with `interrupted`, then `exit`
5	Restart the adapter and resume the thread	The next turn works on the same thread
6	Usage	One or more usage events per turn; counts are not negative; `estimated` matches `capabilities.usage`
7	Prefix stability	Two turns on one thread send identical instructions and tool definitions
8	Context budget	Instructions under 8 KB; 6 tools or fewer

The fake runs the suite in CI on every push. Codex and Claude run it in a nightly canary on the cheapest model, because it needs keys and costs tokens.

Watch for: providers that stream text before they announce the item — create the message on the first delta. Codex reports usage as both a last-turn and a running total; take the last-turn value, never add the totals.

Done when: `kru chat` works the same with `--provider fake`, `codex` and `claude`; the conformance suite passes for all three; and every turn prints its usage.

You can now (example output):

```
$ kru chat --provider claude
> summarise this repository
...
[turn 1 · input 12,410 tokens (10,880 cached) · output 642]
```

Read after: `src/backend/agent-client.ts`, `src/backend/claude-client.ts` (the method switch near line 494 and the functions that emit `item/*`), `src/backend/acp-client.ts`, `src/backend/provider-drivers.ts`, `src/backend/agent/usage-`

`collection.ts`, `scripts/provider-fakes/agents/conformance.ts`, `tools/agent-context/budget.json`.

Improvements over OpenBot

Area	OpenBot	Kru M1
Types	String methods; `params: unknown` read with `getString`	Typed method and event maps; the compiler checks each adapter
Capabilities	Optional fields spread over the client	One declared object
Usage	Collected from provider-specific notifications in the turn lifecycle	A required event; every adapter emits it, exact or estimated
Added context	About 14 KB of instructions and 14 KB of visible tools — roughly 7,000 tokens on every model step, before memories and teammates	Under 8 KB of instructions and at most 6 tools, enforced by a test
Per-turn text	ACP and Muse insert instructions on each normal turn	Fixed at thread start, never per turn
Claude system prompt	Always the full `claude_code` preset with OpenBot's instructions appended	`minimal` or `coding` per agent, with `full` as the baseline; the saving is measured
Conformance	A fake agent per scenario	One suite that every adapter must pass

Implementation record

Code: aminhaiqal/kru; port in `src/providers/protocol.ts`, adapters in `src/providers/{fake,codex,claude}/`, turn consumer in `src/chat/turn.ts`.

Delivered

- **Typed port.** Four methods with checked params and results. Typed notifications for turn boundaries, items, deltas, usage, tool calls and `approval/request`. Every tool or approval request gets exactly one reply, with a deadline and an abort signal. Request ids are never reused, even after a restart.

- **Fake adapter.** `echo`, `stall`, `crash`, `failure` and `tool` modes; deterministic usage values for tests.
- **Codex adapter.** One shared `CodexRuntime` (one app-server) for every agent, with thread ownership that routes events, tool calls and approvals to the right agent. Stopping one agent leaves the others running; the last one stops the process. Uses the experimental dynamic-tool API and `jsonrpcField: false`.
- **Claude adapter.** A custom `systemPrompt`, `settingSources: []` and an explicit built-in tool list. An in-process MCP server for Kru's tools. One persistent streaming input queue per thread. Invented turn ids. Per-turn `usage` kept apart from the session's cumulative `modelUsage`. Resume through the SDK's session metadata.
- **Agent configuration.** `cwd`, `model`, `instructions`, `tools`, `promptProfile` and `permissions` (default `read-only` and `on-request`), also as CLI flags.
- **Prefix control.** Stable profile text, canonical tool order and a snapshot at thread start. `Thread.prompt` reports the profile, instruction bytes, tool count and a SHA-256 prefix hash. Instructions of 8,000 bytes or more and a seventh tool are refused at start and at resume.
- **Approvals.** A missing handler declines; nothing hangs and nothing is granted by default. The CLI declines every request.
- **Fault isolation.** A malformed known notification fails only its turn; three in a row end the process.
- **Chat.** An idle timeout (default 120,000 ms) that resets on activity from the same thread. Usage on stderr after each turn, or `usage: unavailable`.
- **Shutdown.** M0's signal handling for all providers. Claude subprocesses also run in process groups with ownership records.

Changes from the plan

Plan	Built	Why
Rule 3: the adapter emits `turn/completed` with `interrupted` when the process dies	`exit` only; the turn's outcome stays unknown	The adapter cannot know how the turn ended. Rule 3 now says so.
`promptProfile`: `full` or `lean`	`minimal` and `coding`, both short custom prefixes	`full` is a follow-up, needed as the measuring baseline

Plan	Built	Why
At most 6 Kru tools	For Claude, at most 6 built-in and Kru tools together	Leaves no room for M5's tools; a follow-up separates the budgets
A wall-clock turn timeout	An idle timeout	A turn that keeps making progress can run as long as it needs

Verification

- The same conformance cases, including #7 (prefix stability) and #8 (budget), run against all three adapters. Codex runs against a scripted stdio peer; Claude against SDK events and a real in-memory MCP transport.
- Ten repeated offline runs: 1,020 passes, no failures. Strict TypeScript and a frozen install pass.
- Opt-in live tests pass on real Codex and Claude: streaming, usage, resume, tool calls and interruption through the common port.
- `codex-canary.yml` runs nightly at 03:17 UTC against the latest Codex. It checks the `initialize` bytes, dynamic-tool registration and the generated protocol schemas, with no model call. Validated locally against Codex 0.166.0; pinned CI stays on 0.165.0.

Profile benchmark (2026-11-03)

One ordered pair, `full` first, on Claude Sonnet 4.6 (`claude-sonnet-4-6`): repair a TypeScript duration parser with compound h/m/s/ms parts, strict ordering, full-input validation and safe-integer overflow checks. Both profiles passed all 26 independent acceptance checks with unchanged tests.

Measurement	Full	Coding
Input tokens, total (includes cache reads and writes)	161,579	98,257
Cache reads	147,055	66,954
Cache writes	14,515	31,295
Output tokens	8,697	13,896
Input plus output	170,276	112,153 (−34.1%)

Measurement	Full	Coding
Elapsed time	110.7 s	178.8 s (+61.4%)
Cost at list prices, 5-minute cache writes	$0.229	$0.346 (+51%)

Cost uses Sonnet 4.6 list prices per million tokens: $3 input, $15 output, $3.75 for 5-minute cache writes and $0.30 for cache reads (Claude docs). With 1-hour cache writes, `coding` costs 59% more.

Finding: fewer tokens did not mean lower cost. Almost all of the full profile's input was cache reads, the cheapest tokens. The lean profile cut those, but produced more output and twice the cache writes, and took longer. A likely reason, still to test: Claude Code's full system prompt guides the model to work in fewer steps. `full` stays the default profile for coding agents until a controlled benchmark shows otherwise.

Limits: one ordered pair; uncontrolled caching; system prompt, tool list and setting sources changed together. Dollar savings in general and quality parity on other tasks are not measured.

Next benchmark: at least 5 pairs in alternating order with the median and range; a controlled cache (a unique marker for cold starts, or a warm-up run); one change at a time (full prompt with restricted tools, lean prompt with full tools, the current lean profile); model steps and tool calls counted per turn; and a run on a current model.

Follow-ups

- [x] ~~Normalise usage across providers. Claude reports cache reads and writes apart from `input_tokens`; Codex counts cached tokens inside its input. Report `inputTotal`, `cachedRead`, `cacheWrite` and `output`; print the cached part in the CLI; add a conformance check that `inputTotal` is at least `cachedRead + cacheWrite`.~~
- [x] ~~Separate the tool budgets: at most 6 Kru-declared tools; built-in tools chosen by the profile, reported but not capped.~~
- [x] ~~Add the `full` profile (the `claude_code` preset and project settings), then compare `full` and `coding` on one task for tokens and result.~~
- [x] ~~Permission parity: `workspace-write` on Claude allows file tools inside `cwd` and its roots and asks only for shell commands or outside paths. Add a conformance case: the same settings allow and refuse the same actions on every adapter.~~
- [x] ~~Confirm that output or progress from a long shell command resets the idle timer on both adapters.~~

- [x] ~~Merge the canary to the default branch so its schedule runs.~~
- [x] ~~Tag v0.0.3.~~

M2 — The durable mailbox · Done

Status: done on 2026-11-04 and released as v0.0.4. The mailbox stores deliveries and moves them between states; it starts no provider. The implementation record and follow-ups are below the improvements table.

Purpose. M2 makes Kru durable. The mailbox is a SQLite inbox through which all work enters — from a person, another agent or a routine. M2 only stores work and moves it between states; it never runs a turn. Running is M3's job. Keeping the two apart is what lets M7 recover after a crash.

Architecture

- `src/store/db.ts` opens `bun:sqlite` with WAL mode, a busy timeout and foreign keys on, and runs migrations from version 1.
- `src/mailbox/` holds the mailbox API, the delivery state machine and the queue queries.
- `src/agents/` is a minimal agent table, so `enqueue` can check that a recipient exists.
- The CLI adds `kru agent create`, `kru send`, `kru queue` and `kru cancel`.

The mailbox is a port, as the provider is in M1. M4 replaces its internals with the event log without changing a caller.

Data model: messages apart from deliveries

```
CREATE TABLE agents (
  id          TEXT PRIMARY KEY,
  name        TEXT NOT NULL UNIQUE,
  provider    TEXT NOT NULL,
  config_json TEXT NOT NULL CHECK (json_valid(config_json)),
  created_at  TEXT NOT NULL
);

CREATE TABLE messages (
  id              TEXT PRIMARY KEY,
  sender_kind     TEXT NOT NULL CHECK (sender_kind IN
('user','agent','routine')),
  sender_agent_id TEXT REFERENCES agents(id),
  text            TEXT NOT NULL,
```

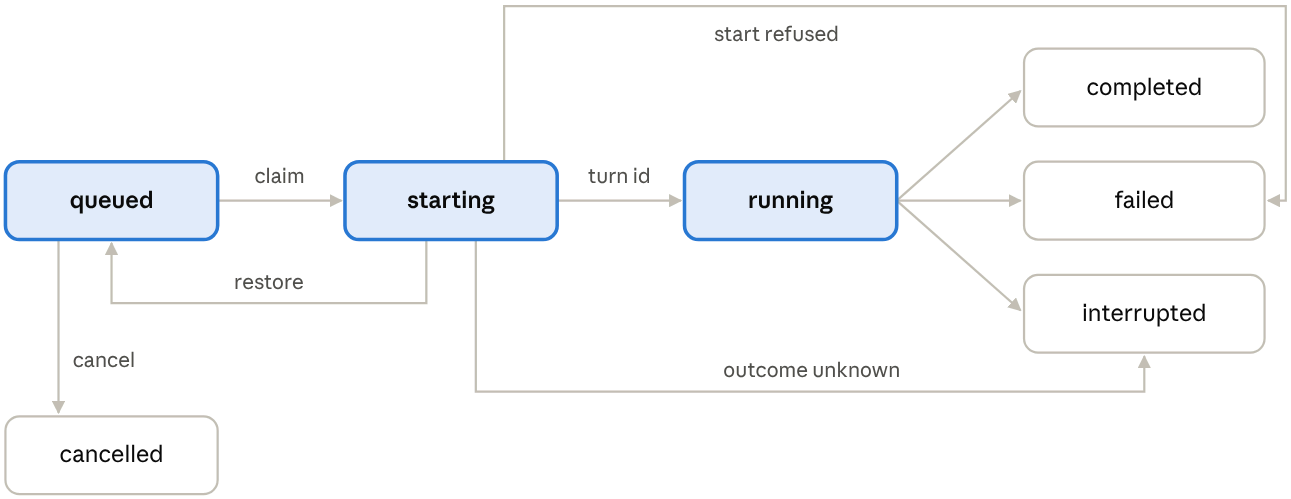
```
  reply_to        TEXT,
  idempotency_key TEXT UNIQUE,          -- NULL allowed; duplicates refused
  request_hash    TEXT,                 -- same key with other content =
client error
  created_at      TEXT NOT NULL
);

CREATE TABLE deliveries (
  seq          INTEGER PRIMARY KEY,     -- queue order; never order by time
  id           TEXT NOT NULL UNIQUE,
  message_id   TEXT NOT NULL REFERENCES messages(id),
  recipient_id TEXT NOT NULL REFERENCES agents(id),
  lane_id      TEXT,                    -- NULL = the agent's main chat
  status       TEXT NOT NULL,
  turn_id      TEXT,
  error        TEXT,
  retry_of     TEXT REFERENCES deliveries(id),
  chain_id     TEXT NOT NULL,           -- M5 hop limit; in the schema from
version 1
  hop          INTEGER NOT NULL DEFAULT 0,
  updated_at   TEXT NOT NULL
);
CREATE INDEX deliveries_queue ON deliveries (recipient_id, lane_id, status,
seq);
```

One message can go to several agents: `send_message` to B and C is one message with two deliveries, each with its own status. OpenBot splits them the same way (`projection_mailbox_messages` and `projection_deliveries`). The auto-increment `seq` gives a strict order; two messages can share a millisecond, and clocks can jump.

The delivery state machine

A delivery only moves forward; final states never reopen



Tinted: open states. Outlined: final states; a retry creates a new delivery.

Delivery state machine · 3 open states, 4 final states

A drain claims a queued delivery into `starting`; a confirmed turn id moves it to `running`. If `turn/start` fails before a turn begins, the delivery is restored to `queued` at its original place, or ends `failed` when the provider refused it. If a turn may have begun, it ends `interrupted` and is never retried automatically.

Mailbox API

```
type DeliveryStatus = "queued" | "starting" | "running" | "completed" |
"failed" | "interrupted" | "cancelled";

interface EnqueueInput {
  sender: Sender;
  recipientIds: string[];
  text: string;
  laneId?: string | null;
  replyTo?: string | null;
  idempotencyKey?: string;
  chain?: { id: string; hop: number };   // set by M5; otherwise a new chain
}

interface EnqueueReceipt { messageId: string; deliveryIds: string[];
duplicate: boolean }

interface Mailbox {
  enqueue(input: EnqueueInput): EnqueueReceipt;
```

```
  claimNext(agentId: string, laneId: string | null): Delivery | null; //
queued → starting
  markRunning(deliveryId: string, turnId: string): void;              //
starting → running
  finish(deliveryId: string, status: "completed" | "failed" | "interrupted",
error?: string): void;
  restore(deliveryId: string): void;                                  //
starting → queued
  cancel(deliveryId: string): void;                                   //
queued → cancelled
  retry(deliveryId: string): EnqueueReceipt;                          //
final → a new delivery
  get(deliveryId: string): Delivery | null;
  listQueue(agentId: string, laneId?: string | null): Delivery[];
  onChange(listener: (change: { deliveryId: string; status: DeliveryStatus })
=> void): () => void;
}
```

Design rules

1. Store and transition only. The mailbox never runs a turn.
2. Every transition is a compare-and-set update, checked against one table of allowed transitions: `UPDATE deliveries SET status = ? WHERE id = ? AND status = ?`. A change count other than 1 means another writer won. This is what lets M3 run several drains without claiming the same work twice.
3. A final state never reopens. A retry creates a new delivery with `retry_of`, so the history shows that the first attempt failed.
4. Idempotency in the same transaction as the insert: a `UNIQUE` key plus a hash of the request. A duplicate returns the first receipt with `duplicate: true`; the same key with other content is refused.
5. Notify after commit. `onChange` fires only after the transaction commits, so M3 never schedules work that was rolled back.
6. Queue order is `seq`, never a timestamp. `restore` keeps the delivery's `seq`, so it does not move to the back of the queue.
7. One writer at a time. WAL mode, `BEGIN IMMEDIATE` for every write and a busy timeout. The API is synchronous, as `bun:sqlite` is, so no mailbox operation can be interleaved halfway through.

8. Migrations from version 1. Each schema change and its `schema_migrations` row commit in one transaction. Every shipped version keeps a fixture, and a parity test checks that a new database and a migrated one have the same schema.
9. Fields for later milestones now: `chain_id` and `hop` for M5; `sender_kind` is the turn cause for M4.

Tests

#	Test	Proves
1	The same key sent twice	One message; the second call returns the first receipt with `duplicate: true`
2	The same key with other text	Refused; nothing written
3	One message to two agents	Two deliveries with independent statuses
4	Restart	Queued rows and their order survive
5	`completed → running`	Refused; the row does not change
6	Two connections call `claimNext` at the same time	They never claim the same delivery
7	`claimNext` on one lane	Takes the lowest `seq` of that lane; other lanes do not change
8	`restore`	Back to `queued` with the original `seq`
9	`cancel`	Works on `queued`; refused on `running`
10	`retry` of a final delivery	A new delivery with `retry_of`; the old one does not change
11	A transaction that rolls back	No `onChange` event
12	Migrations	An empty database and a version-1 fixture end with the same schema
13	An unknown recipient	Refused; nothing written

Done when: all 13 tests pass in ten repeated runs, as in M0 and M1.

Watch for: `UPDATE … RETURNING` needs SQLite 3.35 or later — check the version that Bun bundles. A CLI and the M3 daemon can write at the same time; without a busy timeout, one of them gets `SQLITE_BUSY`.

Optional bridge to M3: `kru deliver --once <agent>` claims one delivery, runs it with the M1 `runChatTurn` and records the outcome. It is the smallest path from message to turn to final state; M3 automates it.

You can now (example output):

```
$ kru agent create reviewer --provider claude
$ kru send reviewer "review the latest commit"
queued  message 7f3c…  delivery d-1
$ kru queue reviewer
d-1  queued  user  "review the latest commit"
```

Read after: `src/backend/mailbox-store.ts` (`enqueue`, `markStarting`, `restoreQueued`, `markTerminal`), the `projection_mailbox_messages` and `projection_deliveries` tables in `src/backend/openbot-database-schema.ts`, "Database migrations" in `src/backend/AGENTS.md`, and `src/backend/openbot-database-schema-parity.test.ts`.

Improvements over OpenBot

Area	OpenBot	Kru M2
Scope	`mailbox-store.ts` is about 3,300 lines: deliveries, attachments, reactions, edits and a file-deletion outbox	Deliveries only; attachments get their own module later
Idempotency	Keys in the store's in-memory state map, pruned by age	A `UNIQUE` column written in the same transaction, and a check that the content matches the key
Transitions	Status checks spread through the store	One allowed-transition table and compare-and-set updates
Retry	The same delivery is requeued	A new delivery that links to the old one
Chain and hop fields	Not part of the design	In the schema from version 1

Implementation record

Code: `src/store/` (opening, migrations, writer transactions), `src/store/migrations/001-initial.sql`, `src/agents/`, `src/mailbox/` (port in `types.ts`, transition table in `state.ts`, SQLite adapter in `sqlite.ts`) and `src/cli/mailbox.ts`.

Delivered

- **Store.** Foreign keys, WAL, a 5-second busy timeout and `synchronous=FULL`. SQLite 3.35 or later is checked at startup (verified on 3.51.0). Every write takes `BEGIN IMMEDIATE` before it reads the state it changes, and public writes refuse to run inside another transaction.
- **CLI.** `agent create` and `agent list`; `send` to one or more agents with `--lane`, `--main`, `--reply-to` and `--expects-reply`; `queue`, `cancel` and `retry`. Agents are named by id or unique name; queue output escapes message text.
- **Transitions.** One allowed-transition table; every change requires exactly one changed row. `seq` uses `AUTOINCREMENT`, so queue positions are never reused. A retry creates a new queued delivery with `retry_of`, keeping the recipient, lane, chain and hop.
- **Idempotency.** A unique key and a SHA-256 hash of the normalised request (sender, sorted recipients, text, lane, reply target, reply flag, chain) in the same transaction as all deliveries. A replay returns the original ids with `duplicate: true`, even after a restart or a retry. Another request under the same key throws `IdempotencyConflictError` and writes nothing. Duplicate recipients are refused.
- **Schema additions.** `sender_routine_id` and `expects_reply`, to match Core types; SQL checks on sender shape, statuses, JSON configuration and key/hash pairs.
- **Notifications.** `onChange` fires after commit, in commit order, never for replays or rolled-back writes. A listener error becomes a warning. Notifications are local hints; SQLite is the source of truth.
- **Migrations.** One migration path for new and existing databases, so no separate latest schema can drift. A frozen v1 fixture and a parity test check schema, retained data and foreign keys.

Changes from the plan

Plan	Built	Why
The diagram had no arrow from `starting` to `failed`	Follows the text, which allows it; the diagram is now fixed	A provider can refuse a start before a turn begins

Plan	Built	Why
A fresh schema plus migrations, checked by a parity test	One migration path for both	Removes the cause of drift instead of testing for it
The sample SQL	Adds `sender_routine_id` and `expects_reply`	Matches the core types
Optional `kru deliver --once`	Left for M3	M3 runs deliveries

Verification

- The numbered cases in `test/mailbox.test.ts` and the v1 parity test in `test/store.test.ts` cover all 13 tests.
- 24 M2 tests run ten times: 240 passes, no failures, including real competing Bun processes on one file.
- Full check: TypeScript passes; 162 tests pass, 9 opt-in tests skipped.

Follow-ups

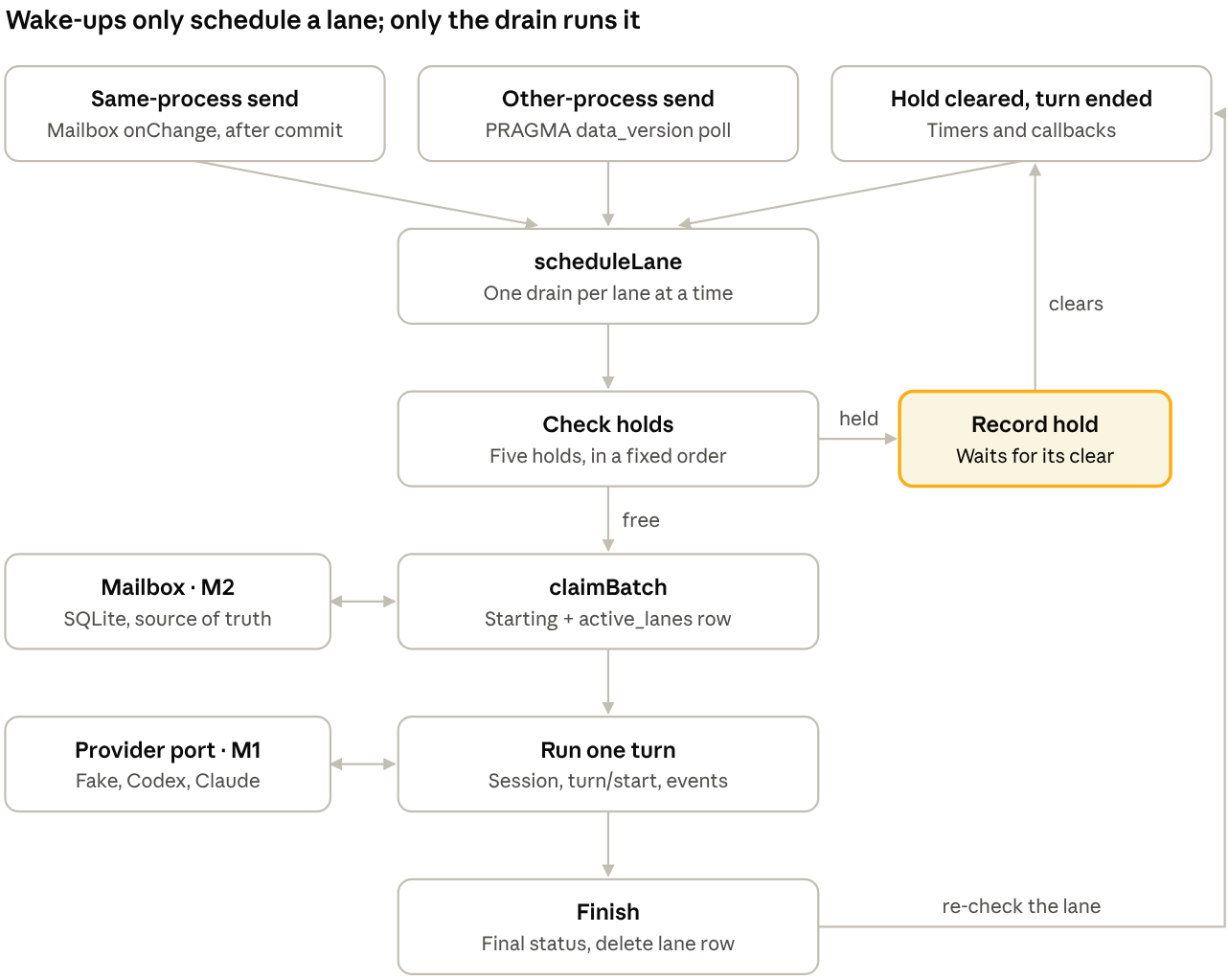
- [x] ~~Find the database the way git finds `.git`: `kru init`, the nearest ancestor `.kru/`, then `KRU_DB` and `--db` (done in M3).~~
- [x] ~~Cross-process wake-ups through `PRAGMA data_version` (done in M3).~~
- [x] ~~One active batch per lane, enforced by SQLite (done in M3).~~
- [x] ~~Re-export the plan under the name Kru.~~
- [ ] Idempotency retention before v0.1: keep keys at least 30 days, pinned while any attempt is open, with no extension on replay. The design is recorded in `m3-design.md`.
- [ ] Input size limits and secret redaction for stored provider errors before v0.1.

M3 — Lanes and the drain scheduler · Done

Status: done on 2026-11-04. Every turn runs an atomically claimed batch of deliveries. The implementation record and follow-ups are below the improvements table.

Purpose. M3 is where Kru starts running work, and the most important milestone: it decides what runs, when, and in what order. A daemon watches the mailbox, picks the next work of each lane, runs it through the M1 port and records the outcome in the M2 mailbox.

Architecture



M3 drain loop · 3 wake sources, 5 steps, 1 hold path

The key idea: wake-ups only schedule a lane, and only the drain runs it. Wake-ups can come from anywhere, any number of times; `scheduleLane` merges them into at most one drain per lane. A held lane records why and waits for that reason to clear.

Module	Owns
`src/daemon/`	`kru daemon`: the single-daemon lock, the boot sweep, wake sources and shutdown
`src/scheduler/scheduler.ts`	`scheduleLane`, the scheduled and draining sets, and the re-check after each drain
`src/scheduler/holds.ts`	Hold reasons, their order, and what clears each one

Module	Owns
`src/scheduler/runner.ts`	One turn: bind the session, start the turn, follow its events, map the outcome
`src/scheduler/slots.ts`	The global cap on concurrent turns, with person-sent work first
`src/scheduler/batch.ts`	Which queued deliveries share a turn, and the combined prompt

The scheduler depends only on the `Mailbox` port (M2), and on the `ProviderClient` port and registry (M1).

New tables (migration v2)

```
CREATE TABLE active_lanes (            -- one turn per lane, enforced by
SQLite
  agent_id   TEXT NOT NULL REFERENCES agents(id),
  lane_key   TEXT NOT NULL,            -- '' = main chat; NULL cannot be in a
primary key
  batch_id   TEXT NOT NULL,
  turn_id    TEXT,
  started_at TEXT NOT NULL,
  PRIMARY KEY (agent_id, lane_key)
);

CREATE TABLE agent_sessions (          -- which native thread continues this
lane
  agent_id         TEXT NOT NULL REFERENCES agents(id),
  lane_key         TEXT NOT NULL,
  provider         TEXT NOT NULL,
  native_thread_id TEXT NOT NULL,
  prompt_hash      TEXT NOT NULL,      -- M1 prefix hash; a change starts a
new session
  updated_at       TEXT NOT NULL,
  PRIMARY KEY (agent_id, lane_key, provider)
);

CREATE TABLE daemon_lock (             -- one daemon per database
  id             INTEGER PRIMARY KEY CHECK (id = 1),
  pid            INTEGER NOT NULL,
  start_identity TEXT NOT NULL,
```

```
  heartbeat_at   TEXT NOT NULL
);

ALTER TABLE deliveries ADD COLUMN batch_id TEXT;
-- agents.config_json gains: paused, dailyTokens
```

`claimBatch` inserts the `active_lanes` row in the same transaction as the `queued → starting` updates, so a second turn in one lane fails at the primary key even if the scheduler has a bug. `agent_sessions` keeps a lane on the same native thread across turns and daemon restarts. When the prefix hash changes, a new native session starts; until M6 adds the handoff, it starts without the earlier conversation.

Wake sources

1. Same process: the M2 `onChange` event, after commit.
2. Other processes, such as `kru send` in another terminal: the daemon reads `PRAGMA data_version` about every 100 ms. It changes only when another connection commits, so each check costs almost nothing.
3. Timers and callbacks: a turn ended, a slot freed, a budget window reset, an agent resumed.

Holds — checked in this order; each names what clears it.

Hold	Checked against	Cleared by
`paused`	`agents.config_json`	`kru agent resume`
`provider-not-ready`	The registry: CLI missing, signed out or starting	The adapter becoming ready
`lane-busy`	An `active_lanes` row for the lane	That lane's turn finishing
`budget`	Today's tokens at or above `dailyTokens` — in memory from M1 usage events, then the M4 ledger	The daily reset, or a higher budget
`slots`	Running turns at the cap (default 2)	Any turn finishing; person-headed lanes go first, then the lowest `seq`

`kru status` shows each held lane with its reason — the answer to "why is my agent doing nothing?"

Batching

`claimBatch` takes the consecutive person-sent deliveries at the head of a lane, up to 10 messages or 32 KB, in one transaction, with a shared `batch_id`. Agent and routine deliveries always run alone, because each needs its own reply context and chain. The combined prompt lists the messages in order with a fixed separator. Batching is the main token saving: each turn resends the agent's whole context, so five messages in one turn cost about one fifth of five turns.

Turn outcome to delivery state — every delivery in a batch gets the same outcome.

What happened	Delivery state
`turn/completed`: completed	`completed`
`turn/completed`: failed	`failed`
`turn/completed`: interrupted by a person	`interrupted`
Provider not ready; `turn/start` never sent	Restored to `queued` at the same place
`turn/start` refused with a definite error	`failed`
`turn/start` timed out with an unknown result	Stays `starting` until `turn/started` arrives, or a 5-minute grace ends → `interrupted`
Process exit or idle timeout during the turn	`interrupted`

Nothing is retried automatically; a person runs `kru retry`.

Daemon lifecycle

- **Start:** take `daemon_lock`. When the lock's process is gone or its start identity does not match (the M0 identity check), take it over; otherwise refuse to start.
- **Boot sweep**, a minimal M7: mark every `starting` and `running` delivery `interrupted` with "daemon restarted", and clear `active_lanes`. Without it, a crash leaves lane rows that block those lanes forever. M7 replaces it with asking the provider what happened.
- **Stop:** stop claiming, wait up to the grace period for running turns, then interrupt them. A second signal force-kills, as in M0.

Design rules

1. Wake-ups only schedule a lane; only a drain runs it.
2. One turn per lane, enforced twice: in memory by the draining set, and in SQLite by the `active_lanes` primary key.
3. Re-check after release. After the draining flag clears, ask the mailbox whether the lane has queued work and schedule it again if so.
4. Every hold names what clears it, and every held lane shows its reason in `kru status`.
5. Every turn has a delivery behind it. A turn that ends with open plan steps does not start another turn.
6. Nothing is retried automatically.
7. One daemon per database.
8. The scheduler imports the `Mailbox` and `ProviderClient` ports only, never an adapter.

Tests

#	Test	Proves
1	Five fast messages to one agent	One running turn, then one batched turn for the other four
2	Two agents with work	They run in parallel
3	A second daemon on the same database	It refuses to start
4	Each hold reason	The lane is held with that reason; clearing it resumes the lane
5	`kru send` from another process	Picked up through `data_version` within 1 second
6	A message sent in the gap before release	It runs — no lost wakeup
7	An agent or routine delivery at the head of a lane	It runs alone, not batched
8	Cap 2, three agents with work	The third waits; a person-headed lane goes first
9	`turn/start` with an unknown result	Stays `starting`; ends `interrupted` after the grace period

#	Test	Proves
10	`kill -9` of the daemon, then a restart	No `starting` or `running` deliveries, no `active_lanes` rows; the queue continues
11	A second `active_lanes` row for the same lane	Refused by the primary key
12	Count turn starts in the fake	Equal to the number of claimed batches — no turn without a delivery

Done when: all 12 tests pass in ten repeated runs.

Watch for: the lost wakeup. A message that arrives while a drain finishes has its wake-up dropped because the lane is still draining, and the drain then ends without looking again. OpenBot's `Effect.ensuring` block in `#drainLane` checks `nextQueued` again after it clears the draining flag; test 6 forces this gap. The same applies to holds: when a budget resets or a slot frees, schedule the held lanes again.

Read after: `src/backend/agent/drain-scheduler.ts` (`#scheduleLane`, `#drainLane`, `startDelivery`, `#settleUnstarted`), `src/backend/agent/turn-slots.ts`, `src/backend/agent/usage-limit-gate.ts`, `src/backend/agent/boot-recovery.ts`.

You can now (example output):

```
$ kru daemon &
$ kru send reviewer "review the latest commit"
$ kru status
reviewer  main  running  turn t-12  batch of 1
coder     main  held     budget (198,400 of 200,000 tokens today)
$ kru agent set coder --daily-tokens 400000
```

Improvements over OpenBot

Area	OpenBot	Kru M3
One turn per lane	In-memory sets only	Also a primary key in SQLite (`active_lanes`)
Wake-ups	One Electron process	Local events plus `data_version`, so the CLI and the daemon can be separate processes

Area	OpenBot	Kru M3
Automatic turns	Up to 3 plan continuations (`PlanContinuation({ limit: 3 })`)	None
Hold reasons	Computed in several places	One table; each reason names what clears it
Concurrency cap	Turn slots on hosted servers only	A global cap by default (2)
Budget	Reacts to the provider's own usage limits	Kru's own daily token budget stops work earlier

Implementation record

Code: `src/daemon/` (composition, lock, session store), `src/scheduler/` (scheduler, holds, slots, batch, session, runner), the batch operations in `src/mailbox/sqlite.ts`, and `src/store/migrations/002-scheduler.sql`. Design decisions are recorded in `m3-design.md`.

Delivered

- **CLI.** `kru init`, `kru daemon`, `kru status` (also `--json`), `agent pause`, `agent resume` and `agent set --daily-tokens`. Daemon flags: `--max-turns`, `--poll-ms`, `--idle-timeout-ms`, `--start-timeout-ms`, `--start-grace-ms` and `--shutdown-grace-ms`.
- **Atomic batches.** `claimBatch` reserves the lane and moves its deliveries to `starting` in one transaction. `active_lanes` has a unique `batch_id` and no row id. `markBatchRunning`, `finishBatch` and `restoreBatch` move every member at once; a finish must match its batch id, so a stale completion cannot release a later batch. `claimNext` still works for M2 callers but cannot bypass a reservation.
- **Batching.** Consecutive person-sent deliveries, up to 10 messages and 32,768 bytes including the separator `\n\n--- Kru message ---\n\n`. An oversized message at the head of a lane fails before any model call, so the messages behind it can run.
- **Wake-ups.** A startup scan, an after-commit change bus shared by every mailbox on one `Store`, and `PRAGMA data_version` every 100 ms, sampled before each scan on one long-lived connection.

- **Holds and slots.** The five holds in the planned order. A global cap of 2; a slot is held through session setup and the unknown-start grace. Person-headed lanes go first, then the lowest `seq`.
- **Budgets.** A start gate: a running turn can take an agent past its limit. Counted from each turn's latest `last.totalTokens`, as a difference. Kept in memory; reset at UTC midnight or a daemon restart.
- **Sessions.** Bindings keyed by agent, lane and provider, with the M1 `prompt_hash` and a `config_hash` (prefix, provider, model, cwd, permissions). An unchanged binding is reused; after a restart the runner tries `thread/resume`. A setup failure restores the batch and holds the provider without sending `turn/start`.
- **Outcomes.** As in the table above. Listeners are installed before `turn/start`; a late `turn/started` confirms the original batch. Output, progress and tool activity reset the idle timer; a running native command pauses it. When an outcome stays unknown, the runner force-stops the provider runtime before releasing the lane.
- **Daemon.** The lock uses PID and the M0 start identity, with a heartbeat every second and a status snapshot for `kru status`. A live matching PID always keeps the lock. The boot sweep marks `starting` and `running` deliveries `interrupted`, clears reservations and discards their session bindings. Shutdown waits up to 5 seconds, then interrupts; a second signal force-stops.

Changes from the plan

Plan	Built	Why
`daemon_lock` with a heartbeat	Also a status snapshot	Lets another CLI process show hold reasons and token totals
`agent_sessions` with `prompt_hash`	Also `config_hash`	A model, cwd or permission change also starts a new session
An unknown outcome ends `interrupted`	Also force-stops the provider runtime	Stops an unknown turn from running on unseen
Budget counted from usage events	Counted from the latest snapshot as a difference	Repeated usage events are not added twice

Verification

- The 12 numbered tests in `test/scheduler.test.ts`, with real CLI sends from another process, competing batch claims, a refused second daemon, a forced lost-wakeup gap and a real `SIGKILL` and restart.
- 43 scheduler tests run ten times: 430 passes, no failures. Frozen v1 and v2 fixtures check migration parity and rollback.
- Full check: TypeScript passes; 203 tests pass, 9 opt-in tests skipped. No model or API key is needed.

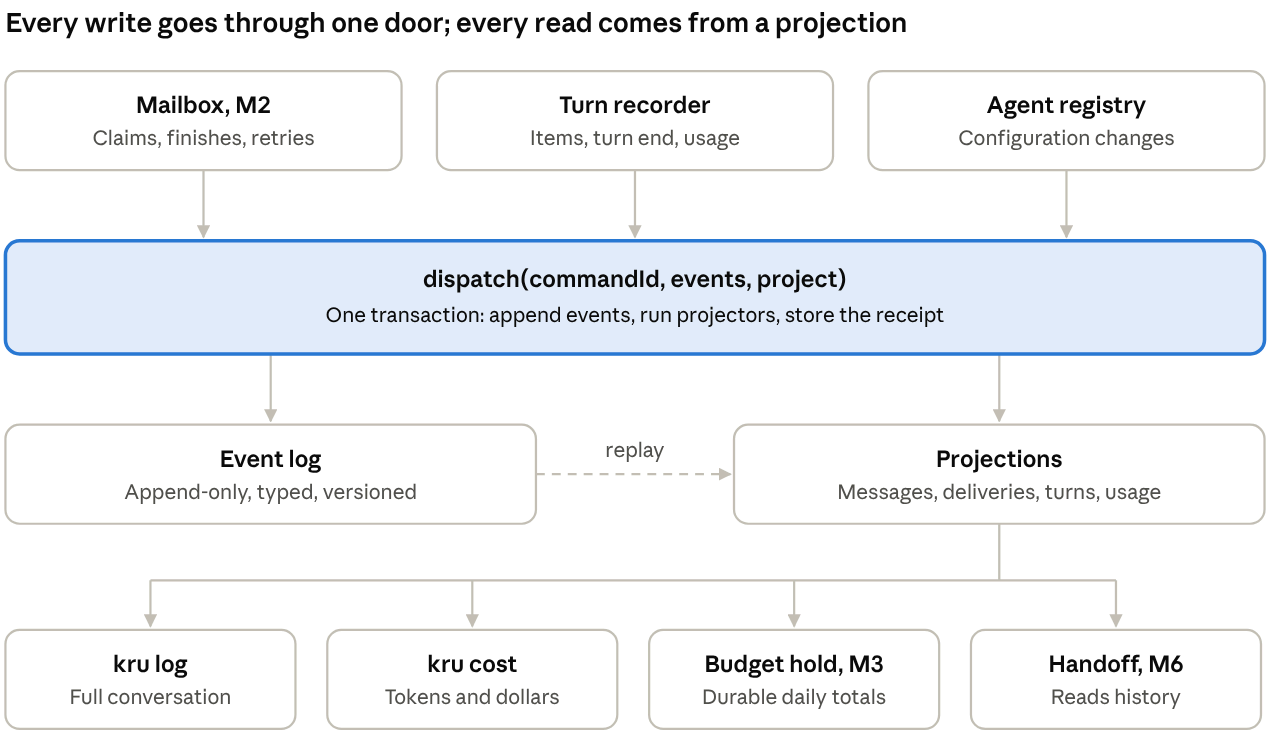
Follow-ups

- [x] ~~Targeted recovery for an unknown Codex turn: abandoned threads get tombstones, and a liveness probe restarts the shared process only when it stops answering, so healthy agents are not interrupted. Claude keeps per-thread process termination.~~
- [x] ~~Heartbeat every 10 seconds; the status snapshot is written only when it changes.~~
- [x] ~~Durable budget counters — moved to M4, where the cost ledger provides them.~~
- [x] ~~Budget days follow local time, with a `--budget-timezone` override.~~
- [x] ~~The boot sweep keeps session bindings and tries resume before replacing a missing cache.~~
- [x] ~~Mailbox adapters and agent registries on one `Store` publish on one change bus, so configuration changes wake the scheduler.~~
- [x] ~~Tag v0.0.5.~~

M4 — Event log and projections

Purpose. M4 gives Kru a memory. Everything that happens becomes an event in an append-only log, and the tables Kru reads become projections of those events. M4 also adds the cost ledger: every turn records its tokens and its dollars. The M1 benchmark showed why dollars matter: the lean profile used 34% fewer tokens but cost about 51% more at list prices, because it produced more output and more cache writes.

Architecture



M4 write and read paths · 3 writers, one dispatch, 4 readers

Every write goes through one door, `dispatch`, and every read comes from a projection. When a projection is wrong or its schema changes, replaying the log rebuilds it.

Domain state and operational state

Event sourcing makes every write two writes and the log only grows, so M4 splits Kru's state in two:

Kind	Examples	Event-sourced
Domain state: what happened and what it cost	Messages, delivery transitions, turns, items, usage, agent configuration	Yes; replay rebuilds it
Operational state: who holds what now	`active_lanes`, `daemon_lock`, `agent_sessions`	No; plain tables that the boot sweep resets

Module	Owns
`src/store/events.ts`	`dispatch`, the event table, command receipts, typed and versioned payloads

Module	Owns
`src/store/projectors/`	One projector per projection (mailbox, conversation, usage), each a function from an event to table writes
`src/turns/recorder.ts`	Provider events from the M3 runner; live deltas from memory; items and turn ends written as events
`src/cost/prices.ts`	The price table and the cost of each turn
`src/cli/`	`kru log`, `kru cost` and `kru replay --verify`

Events

Event	Written by	Updates
`message.enqueued`	Mailbox	`messages`, `deliveries`
`delivery.claimed`, `running`, `restored`, `finished`, `cancelled`, `retried`	Mailbox	`deliveries`
`turn.started`	Recorder	`turns`: agent, lane, batch, provider, model, profile, prefix hash
`item.completed`	Recorder	`thread_messages`: assistant text, tool calls, commands, file changes
`turn.completed`	Recorder	`turns`: status, error, duration
`usage.recorded`	Recorder	`usage_records`: tokens, cost, price id, cause, chain
`agent.configured`	Registry	`agents`

Message deltas are never events; there are too many. They stream from memory to live listeners such as `kru log --follow`, and only the final text of each item is stored. Optionally, the recorder saves a partial snapshot about every 2 seconds to a scratch table, so another process can show live text and a crash leaves the partial text for M7.

A turn ends in one transaction

When a turn ends, one `dispatch` writes `delivery.finished` for each batch member, the remaining `item.completed` events, `turn.completed` and `usage.recorded`, and removes the lane reservation. All of it commits or none of it does: no delivery is ever `completed` without its answer or its cost. `dispatch` becomes the only transaction owner, and the M3 batch operations become commands that run inside it, as the M2 notes anticipated.

The cost ledger

```
CREATE TABLE prices (
  id TEXT PRIMARY KEY, provider TEXT NOT NULL, model TEXT NOT NULL,
  input_usd REAL NOT NULL, cache_write_5m_usd REAL, cache_write_1h_usd REAL,
  cache_read_usd REAL, output_usd REAL NOT NULL,      -- per million tokens
  effective_from TEXT NOT NULL, source_url TEXT NOT NULL
);

CREATE TABLE usage_records (
  turn_id TEXT PRIMARY KEY, agent_id TEXT NOT NULL, lane_key TEXT NOT NULL,
  cause TEXT NOT NULL, chain_id TEXT, provider TEXT NOT NULL, model TEXT NOT
NULL,
  input_total INTEGER NOT NULL, cached_read INTEGER NOT NULL,
  cache_write INTEGER NOT NULL, cache_write_ttl TEXT, output INTEGER NOT
NULL,
  estimated INTEGER NOT NULL, price_id TEXT REFERENCES prices(id),
  cost_usd REAL,                      -- NULL when no price is known, never 0
  recorded_at TEXT NOT NULL           -- UTC
);
CREATE INDEX usage_by_agent_time ON usage_records (agent_id, recorded_at);
```

- Cost is computed when the turn is recorded and stored with its price id, so a later price change never rewrites history.
- An unknown price stores `NULL`. `kru cost` shows unpriced turns on their own line and never counts them as free.
- The cache-write lifetime is stored: a 1-hour cache write costs about 60% more than a 5-minute one.
- Budgets come in tokens and dollars, and the M3 budget hold checks both. Daily totals are a range query on `recorded_at` from the start of the local day, so a change of budget timezone never moves past usage to another day. Totals load from the table at startup, which ends the reset on restart.

Migration v3

The database already holds M2 and M3 messages and deliveries. Migration v3 writes genesis events (`message.imported`, `delivery.imported`) from the current rows, in `seq` order, so a replay reproduces today's state. Its test migrates the frozen v2 fixture, replays every event into an empty database and compares every projection row by row.

Design rules

1. `dispatch` is the only transaction owner. Projectors never open a transaction; OpenBot's `src/backend/AGENTS.md` states the same rule and what breaks without it.
2. Projectors are deterministic: no clock, no random ids, no network. Every value they write comes from the event payload.
3. Domain state is event-sourced; operational state is not.
4. A turn ends in one `dispatch`: final statuses, items, turn end, usage and lane release.
5. Message deltas are never events.
6. Shipped event shapes are frozen, like migrations. A change is a new payload version plus an upcaster that converts old events when they are read.
7. Command receipts make commands idempotent: a repeat returns the stored result and appends nothing.
8. Cost is stored with its price id; an unknown price is `NULL`, never 0.
9. Budgets are in tokens and dollars, counted by UTC timestamp ranges.

Tests

#	Test	Proves
1	`kru replay --verify` on a busy database	Every projection rebuilds identically
2	Replay into two empty databases	Both are identical, timestamps included
3	A command run twice	The stored result returns; no events are appended
4	A failure injected halfway through a turn end	Nothing commits: no status, items, usage or lane release
5	Daemon restart	`kru log` shows the full conversation
6	`kru cost` totals	Equal the sum of the usage events
7	A model with no known price	`cost_usd` is `NULL`; the turn shows as unpriced

#	Test	Proves
8	A price change after a turn	That turn's cost does not change
9	Daemon restart with a spent budget	The budget stays spent, in tokens and in dollars
10	A budget timezone change	Past usage stays on its day
11	An event stored in an old payload version	Its upcaster reads it
12	The frozen v2 fixture	Migrates, replays and matches row by row

Done when: all 12 tests pass in ten repeated runs.

Watch for: usage can arrive several times for one turn; store the turn's final value, never the sum. Import genesis events in `seq` order, or replay rebuilds a different queue.

Read after: `src/backend/database/database-core.ts` (`dispatch`), `src/backend/database/thread-replay.ts`, `src/backend/agent/delta-buffer.ts`, `handleNotification` in `src/backend/agent/turn-lifecycle.ts`, `src/backend/agent/usage-collection.ts`, `src/backend/agent/usage-pricing.ts`, `src/backend/database/agent-usage.ts`, and the transaction rules under Size in `src/backend/AGENTS.md`.

You can now (example output):

```
$ kru log reviewer --follow
$ kru cost --by agent --since 7d
agent      turns   input     cached    output   cost
reviewer      42   510,200   431,900   38,120   $1.12
coder         17   205,400   160,010   11,300   $0.61
unpriced       2                                 n/a
$ kru replay --verify
replayed 1,284 events; 6 projections match
```

Improvements over OpenBot

OpenBot uses the same core design — `orchestration_events`, projection tables and command receipts in one transaction — so this part learns from it directly. The differences:

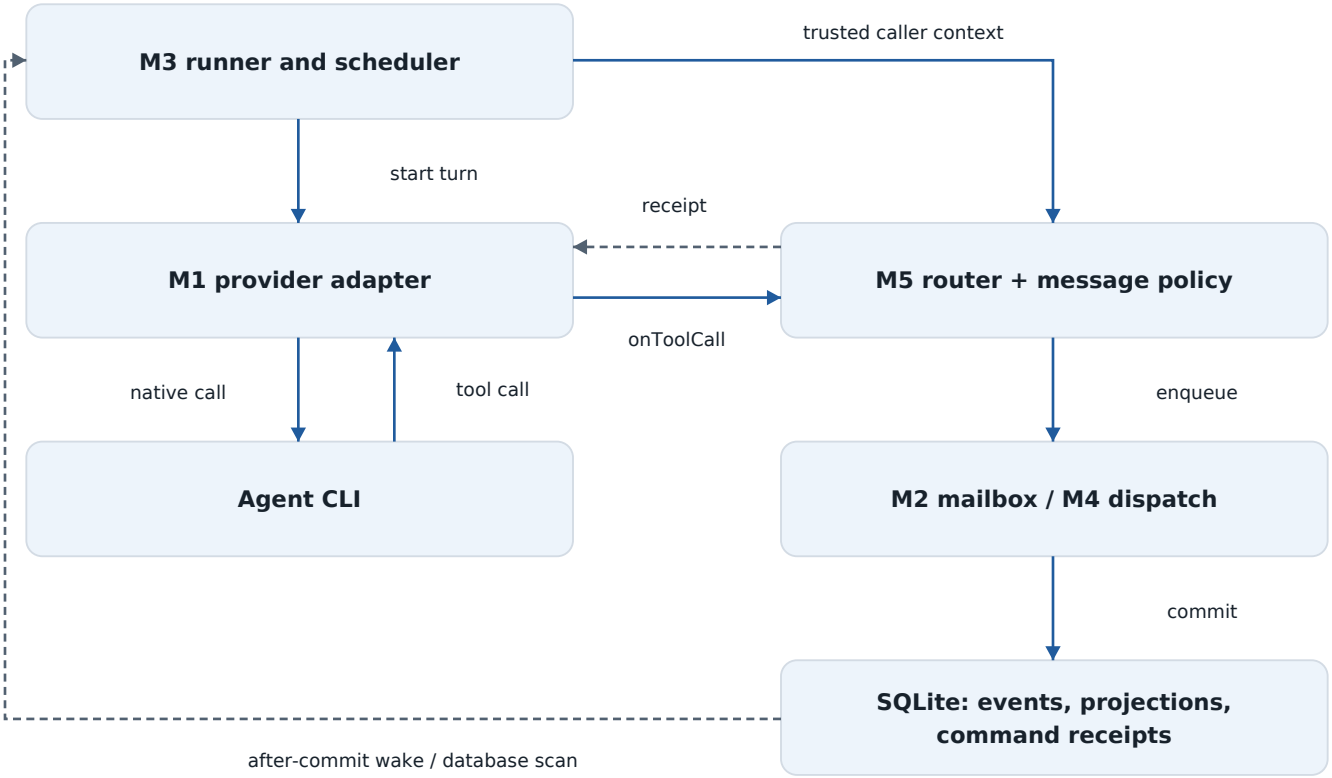
Area	OpenBot	Kru M4
Replay check	Rebuilds one thread's projection (`ThreadReplay`)	`kru replay --verify` rebuilds every projection and compares
What the log holds	Not stated as a rule	An explicit split between domain and operational state
Cost	Token counts, with a price table used for estimates (`usage-pricing.ts`)	A stored cost per turn with its price id; unpriced turns shown, never $0; dollar budgets
Event changes	`schema_migrations` covers tables	Payload versions and upcasters cover events too
Existing data	Built with the event log from the start	Genesis events import earlier rows, checked by a replay test

M5 / 01

Tools and agent-to-agent messages

Status: planned implementation specification. Expanded 4 November 2026. This section records the architecture agreed for M5; it is not an implementation record. Inspect the current M1-M4 interfaces before coding.

Purpose. Let agents hand work to each other through the durable mailbox. A tool call accepts work and returns a receipt; the existing scheduler decides when the recipient runs. Every resulting turn has a delivery behind it.



Scope and ownership

M5 exposes only send_message and list_agents. The adapter translates native calls; the router validates the caller and selects a handler; the message service applies reply and chain policy; M2/M4 persist; M3 schedules. Adapters never write SQLite, and message handlers never start recipient turns.

Dependencies: M1 tool-call bridge, M2 durable messages and deliveries, M3 lanes and batch execution, M4 dispatch, command receipts and usage ledger. M6 history search, provider handoff and v1.0 tool discovery remain outside M5.

M5 / 02

Tool contracts and trusted context

send_message: explicit requests and replies

```
type SendMessageArgs =
  | { kind: "request"; recipientIds: string[];
      text: string; expectsReply?: boolean }
  | { kind: "reply"; replyToMessageId: string;
      text: string };

type SendMessageReceipt = {
  messageId: string;
  deliveryIds: string[];
  duplicate: boolean;
};
```

A request with expectsReply: false is informational; false is the default. A reply derives its recipient and lane from the stored request. It never accepts a destination override and always sets expectsReply to false. A receipt means accepted durably, not completed by the recipient.

list_agents: a small directory

```
type AgentSummary = {
  id: string; name: string;
  description?: string; paused: boolean;
};
```

Return a deterministic ordering and only metadata needed to choose a recipient. Do not expose credentials, raw provider configuration or private session IDs. A paused agent can receive queued work; its pause prevents execution, not enqueue.

The runner supplies authority

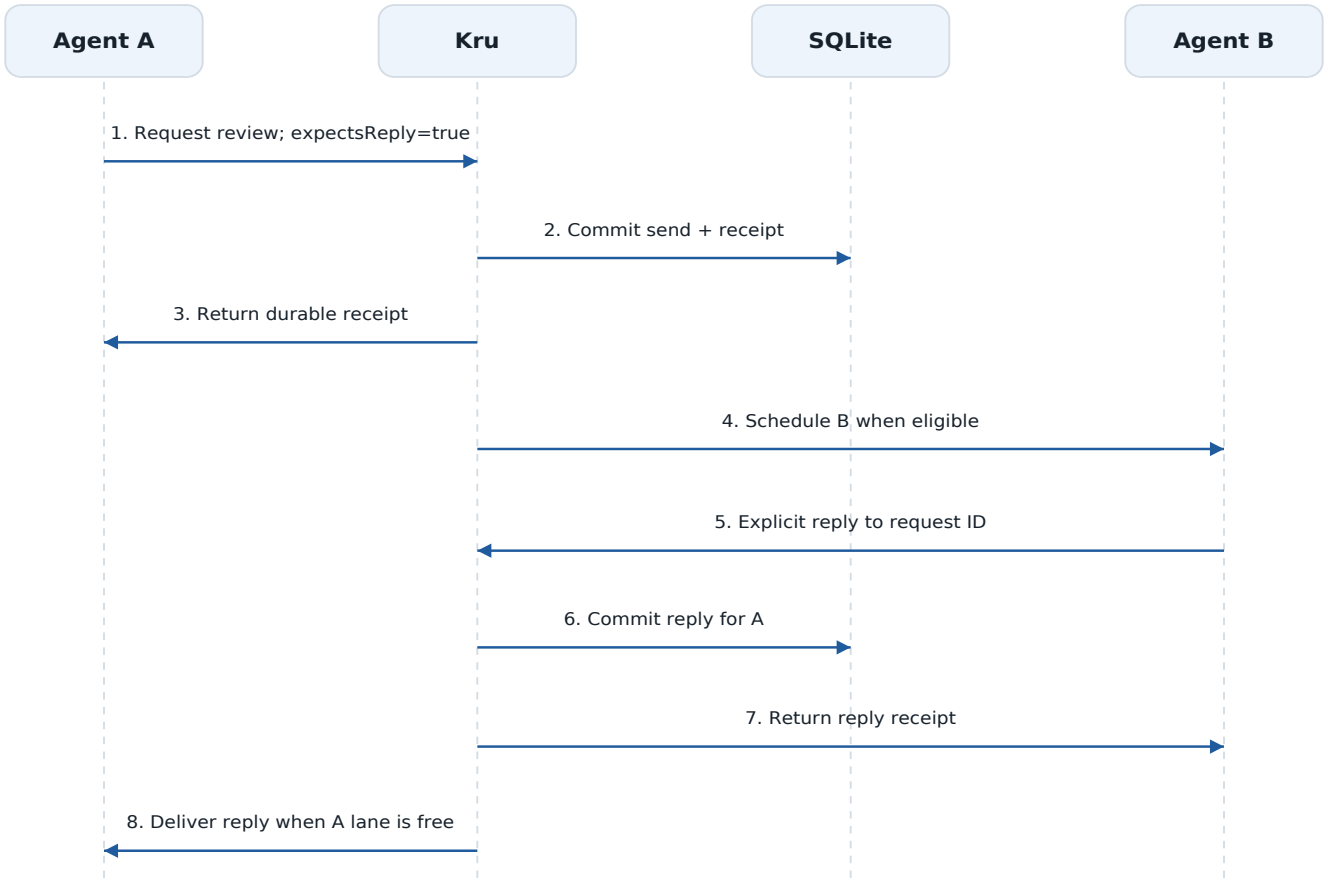
```
type ToolExecutionContext = {
  agentId: string; laneId: string | null;
  executionId: string; // durable Kru-owned turn or batch ID
  providerThreadId: string; providerTurnId: string;
  sourceDeliveryIds: string[];
  chainId: string; hop: number;
  signal: AbortSignal;
};
```

Resolve this context from the active execution and provider ownership mapping. The model cannot set the sender, execution identity, idempotency key, chain, hop or return lane. Verify ownership even when a provider process serves several agents. Use a stable adapter call ID for redelivery of the same invocation.

Validate strict argument shapes, known non-duplicate recipients, non-empty text and configured size limits. A request targets recipients' main lanes in M5; a reply returns to the originating lane. Validate the rendered delivery envelope against the runner's input limit before committing.

M5 / 03

Asynchronous exchange and reply routing



A continues after receiving its send receipt. B may start before A finishes if a global slot is available. A reply that arrives while A is busy waits in A's lane. With one slot, A finishes, B runs, and A then processes the reply: no waiting turn holds the slot.

One reply path

B explicitly calls send_message(kind: "reply"). Kru does not also forward B's final assistant text. This prevents accidental duplicate answers and makes reply intent inspectable. B can reply during its turn; its subsequent turn failure does not retract the committed reply.

Origin and destination

Persist the source execution and originating agent lane on the outgoing message, directly or through a durable event-backed relationship. Replies resolve these values from the stored request. Do not infer a return lane from the sender's current session or default everything to main.

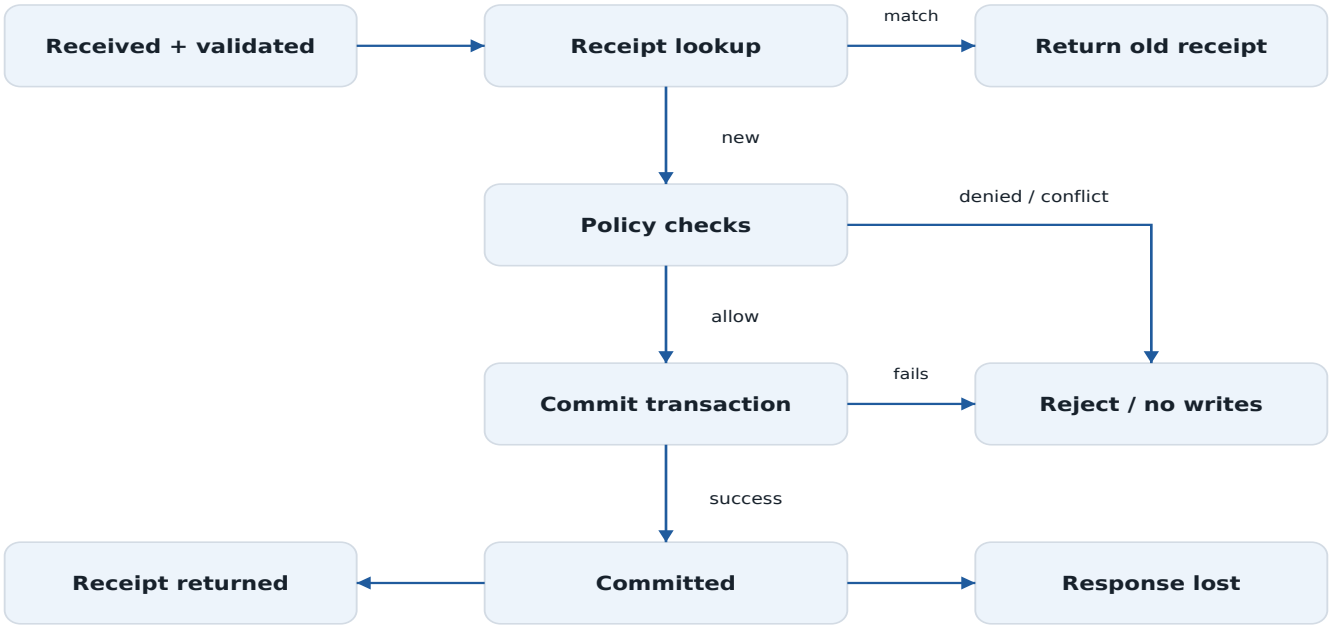
No automatic continuation

Finishing a turn, leaving an unanswered request, or having an open plan step does not create another turn. Only a queued delivery does. No automatic retry is added by M5; unknown outcomes remain interrupted for a person to inspect.

M5 / 04

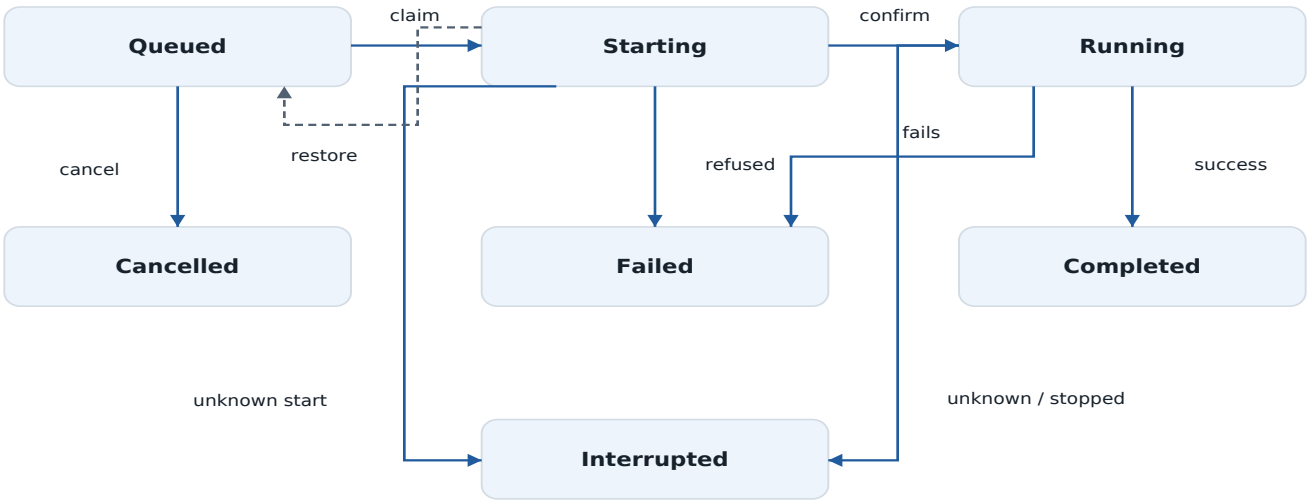
Tool-call and delivery states

Keep tool-call processing, delivery execution and reply expectation separate. Tool stages below are logical processing stages; a database row is not required for every stage.



Lost response: replay the same invocation to return its stored receipt.

Recipient delivery: reuse M2/M3



Terminal deliveries never reopen; manual retry creates a new delivery.

Cancellation before the commit boundary rejects a new send. After commit, the accepted delivery survives cancellation or a lost tool response. A definite pre-start failure can restore queued work; an unknown execution outcome must not be automatically retried.

M5 / 05

Reply policy and recipient context

Enforce reply rules in the engine

• A reply must reference a request delivered to the replying agent in the same causal chain. The request must have expectsReply=true.

• Derive the target agent and originating lane from that request. Set the reply's expectsReply=false and preserve its chain.

• Allow one committed reply per request message and responding agent. Enforce uniqueness transactionally, so distinct tool calls or retried deliveries cannot create a second answer.

• A broadcast request to B and C allows one reply from each. Replying to a reply, or to an informational message, returns a clear tool error.

• A new request is still possible, but inherits the current chain and hop. The model cannot evade propagation limits by relabelling a reply as a request.

Reply tracking is derived, not a delivery status

Condition	Meaning
Not requested	The message has no reply obligation.
Requested, no reply recorded	The request remains unanswered, even if the recipient turn completed.
Reply committed	An answer exists and its delivery is queued, running or terminal.
Reply delivery completed	The original sender processed the answer; this does not certify answer quality.

Rendered delivery envelope

```
From: coder
Message: msg_123
Reply requested: yes

Please review the authentication changes.

To answer, call send_message with:
kind: reply
replyToMessageId: msg_123
```

The envelope helps the model act. Stored records and trusted execution context remain the authority. Keep generic tool instructions and tool definitions stable at thread start; put message-specific content in the turn input. Register the two tools through each adapter's existing tool mechanism.

M5 / 06

Atomic sends, idempotency and failures

Commit the send while the sender turn is running

One M4 dispatch command writes the message event, message and delivery projections, reply claim (when applicable), chain accounting updates and stored receipt. All commit together. dispatch remains the only transaction owner; mailbox and projectors must not open nested transactions.

Publish wake-up hints only after commit. The M3 database scan remains a fallback when a notification is lost. Never wait for turn completion to commit a send: the recipient may already be executing by then.

Stable invocation identity

```
key = hash([
  "kru.send_message.v1",
  executionId,
  callId,
]);
```

Use a canonical, unambiguous encoding. executionId is Kru-owned and durable, reusing the existing M3/M4 turn or batch identity where suitable. This refines the original threadId + turnId + callId proposal so native session replacement cannot collide. Store a hash of the normalised request alongside the receipt.

Case	Required outcome
Same key and same content	Return original IDs, duplicate=true. Append no new events or deliveries.
Same key, different content	Return IDEMPOTENCY_CONFLICT; write nothing.
Transaction rolls back	No partial message, recipient deliveries, reply claim or receipt.
Commit succeeds; response lost	Keep accepted work. Replaying the invocation returns its receipt.
Sender turn later fails	Already committed sends remain valid.
New call after turn ends	Reject TURN_NOT_ACTIVE. A valid replay may return its old receipt without new work.

Validation order and error contract

Validate shape and trusted caller identity, then look up the receipt. A matching committed receipt is returned before reapplying mutable policy limits. For a new invocation, enforce active-turn, reply, size and chain checks in the same atomic command that enqueues.

Return structured ToolResult errors with a stable code and actionable message: INVALID_ARGUMENT, UNKNOWN_RECIPIENT, TURN_NOT_ACTIVE, REPLY_NOT_ALLOWED, ALREADY_REPLIED, MAX_HOPS_EXCEEDED, CHAIN_LIMIT_EXCEEDED and IDEMPOTENCY_CONFLICT. The adapter converts these to native tool errors. Preserve the existing abort and exactly-one-response contract.

Idempotency covers a repeated invocation, not semantic duplicates created by separate call IDs. Do not automatically rerun a provider turn to recover a missing tool response. Keep receipts for at least the mailbox retention window and while related attempts remain open.

M5 / 07

Chains, limits and cost attribution

Propagation rules

```
outgoing.chainId = context.chainId;
outgoing.hop = context.hop + 1;

if (outgoing.hop > chain.maxHops) reject();
if (chain.agentDeliveries + newRecipients >
    chain.maxDeliveries) reject();
```

Root work starts at hop 0. The default maxHops is 4: hops 1-4 may be accepted; attempting hop 5 returns a tool error and creates no deliveries. Replies also increment the hop. The limit prevents further propagation; already queued work in the chain is not silently cancelled.

Depth is not total work. Add a finite per-chain delivery cap to contain fan-out. The M5 design default is maxDeliveries=32 agent-originated recipient deliveries, including replies. A broadcast to three recipients consumes three units. Persist policy on the chain; only user configuration can set it, never tool arguments. This default is an initial engineering choice, not a benchmark result.

Enforce the cap atomically against durable projections, counting accepted deliveries even if subsequently cancelled. Duplicate invocation replay consumes no units. Manual retries preserve chain and hop and count as additional attempts; reject a retry that would exceed the chain cap. Root user deliveries are not included in this agent-propagation allowance.

Batching user messages without double-counting cost

Agent-originated deliveries run alone, as M3 already specifies. A single user delivery retains its root chain. For a multi-message user batch, allocate one durable execution chain and retain links to all original delivery IDs and root chains. The batch execution and its descendants use that execution chain. Do not rewrite original messages to pretend their chains were always identical.

Record each turn's final usage and cost once against its execution chain. Show batch-origin links in chain reporting; do not split tokens evenly across messages or count the same turn again under each origin. Explicit user retries retain the selected execution lineage rather than creating a fresh automatic allowance.

Reporting and guarantees

kru cost --by chain shows chain ID, turns, input/cache/output tokens, priced cost and unpriced turns. Keep M4's stored price IDs and NULL for unknown prices. Report final usage snapshots, not the sum of repeated usage notifications.

Hop and delivery limits bound propagation. They do not guarantee a dollar ceiling: a single turn can be expensive, and the existing daily budget gate can be exceeded by a running turn. M5 reports observed cost and retains the M3/M4 budget holds; hard in-flight spend enforcement is separate scope.

M5 / 08

Storage, modules and implementation order

Durable records and projections

Data	M5 treatment
Existing mailbox fields	Reuse sender_agent_id, reply_to, expects_reply, idempotency_key, request_hash, chain_id and hop.
Message origin	Persist source execution, source delivery IDs and sender lane. These must survive provider session replacement.
Reply claim	Unique (request_message_id, responding_agent_id), pointing to the committed reply message.
Chain projection	Root/execution links, snapshotted maxHops/maxDeliveries and accepted propagation/attempt counts.
Command receipt	Reuse M4's receipt mechanism; avoid a separate competing tool idempotency store.
Usage ledger	Assign each turn once to its execution chain; preserve M4's pricing and unknown-cost semantics.

Use a new migration after the actual current schema version. Do not edit shipped migrations or event shapes. Add versioned message-origin/chain events or upcasters as needed. Existing messages without a recoverable return lane must not guess a side lane; expose an explicit unsupported legacy reply error. Rebuild and verify all new domain projections from the event log.

Module ownership

```
src/tools/
  definitions.ts     Stable tool schemas and instructions
  router.ts          Dispatch, caller checks, error boundary
  context.ts         Trusted execution context
  send-message.ts    Reply/chain policy; mailbox command
  list-agents.ts     Minimal agent directory
src/store/projectors/
  ...                Message origin, replies, chains
test/tools/
  router.test.ts
  messaging.test.ts
  messaging-e2e.test.ts
```

Build order

1. Inspect M1-M4 contracts; define typed arguments, results and error codes.
2. Add versioned events, migration and deterministic projections.
3. Implement atomic send policy and reply uniqueness through dispatch.
4. Wire both tools into the runner and existing provider bridge.
5. Render delivery envelopes and expose unanswered requests.
6. Add chain reporting and deterministic fake-provider end-to-end tests.
7. Run adapter integration checks and the v0.1 release checklist.

M5 / 09

Acceptance tests and release outcome

Run these cases against deterministic fake providers first. Use the shared tool bridge so the engine tests need no API key. Supplement with the existing Codex/Claude adapter integration checks.

#	Test	Proves
1	A requests B; B explicitly replies; A processes it later.	One asynchronous mailbox execution path.
2	Repeat the exchange with one global slot.	No sender wait or slot deadlock.
3	Reply reaches A while its originating side lane is busy.	Correct lane; queued until that lane is free.
4	Repeat a call before/after restart; change its arguments.	One enqueue; stable IDs; conflict on changed content.
5	Lose tool response after commit, then replay invocation.	Accepted send survives; replay consumes no quota.
6	Inject failure partway through a multi-recipient send.	No partial event, delivery, claim, quota or receipt.
7	Interrupt or fail the sender after a committed send.	Child work is retained; no implicit rollback.
8	Try wrong caller ownership, malformed args and a late new send.	Clear errors; no unauthorised or stale enqueue.
9	Reply to an informational message or reply; race two answers.	Reply policy and one answer per responder enforced.
10	Force A/B acknowledgements and fresh-request loops.	Reply loops refused; fresh sends inherit the chain.
11	Accept hop 4; attempt hop 5.	Boundary is exact; rejected send adds no deliveries.
12	Race fan-out sends at the chain cap; cancel and manually retry.	Atomic limit; no refund on cancel or reset on retry.
13	Batch user messages from different roots, then delegate.	One execution chain; origin links; cost counted once.
14	Complete B without replying; inspect request status.	Completed delivery is visibly different from answered.
15	Replay new projections; migrate existing mailbox data.	Deterministic state, preserved history and lane policy.
16	Compare chain totals with final M4 usage records.	Correct cost/tokens; unknown prices remain visible.

Done when: all 16 cases pass in ten repeated runs; type checks and relevant existing suites pass; two fake agents demonstrate the exchange without an API key. Confirm both real adapters register and round-trip the two tools through their existing integration checks.

Read after building: OpenBot handle and #sendMessage in src/backend/agent/openbot-tool-router.ts; src/backend/agent/tool-call-idempotency.ts; src/backend/agent/delivery-content.ts. Follow the project's independent implementation rule.

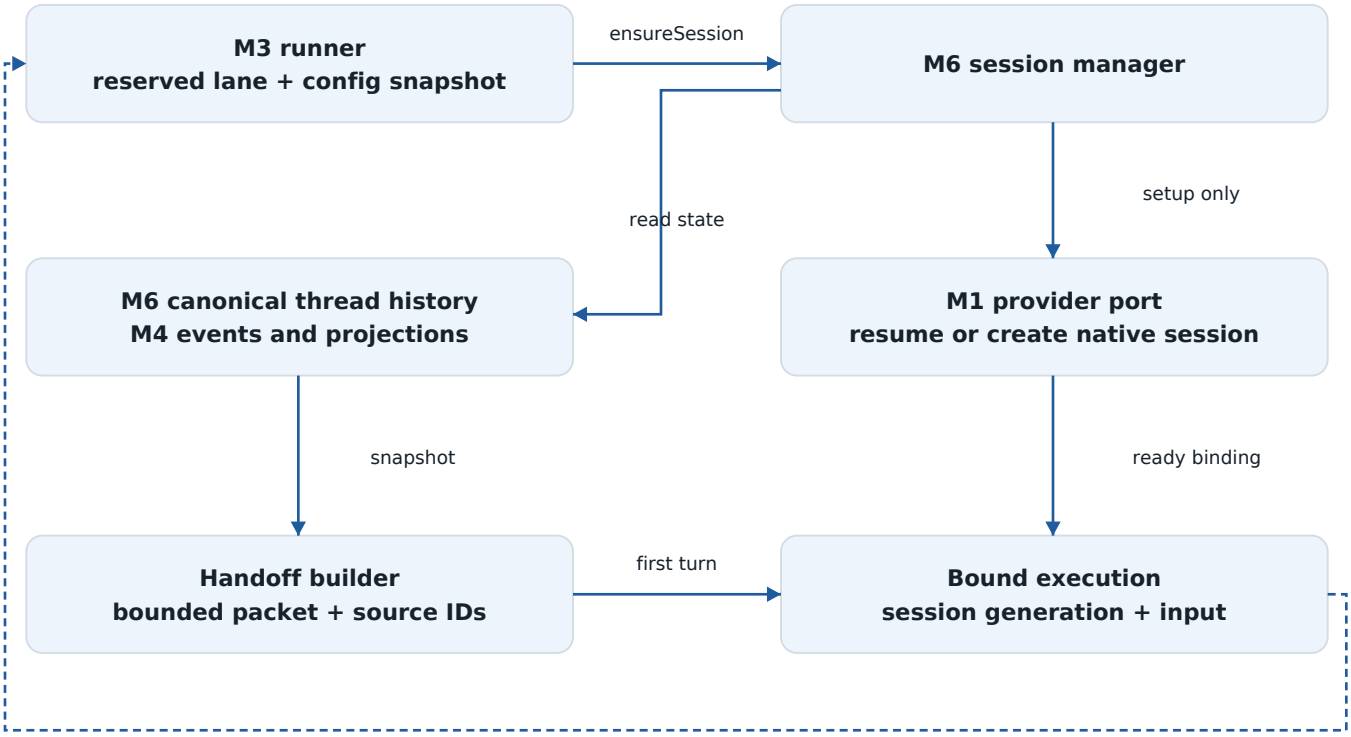
You can now: run a coder and reviewer that hand work to each other with durable receipts, bounded propagation, explicit replies and visible exchange costs. Complete the shared release checklist and release v0.1.

M6 / 01

Threads apart from provider sessions

Status: planned implementation specification. Expanded 4 November 2026. This section defines the M6 architecture and acceptance criteria; it does not claim the implementation exists. Confirm the current repository interfaces before coding.

Purpose. Keep a lane's conversation identity and recorded history when an agent changes provider or loses its native session. Kru owns the thread. The provider owns a disposable execution session. A new session receives a bounded handoff; older evidence remains available through history_search.



Return to M3: only the runner sends turn/start for a claimed delivery.

One execution path, with a session preparation step

The M3 runner reserves the lane, snapshots configuration and asks M6 to ensure a session. M6 resolves the permanent thread, resumes or creates a native session and prepares input. It returns to M3; only the runner sends turn/start. Session preparation must never perform a hidden model turn.

Scope: durable thread identity; versioned session generations; replacement reasons; deterministic handoff packets; scoped history_search; migration from agent_sessions. Automatic compaction, cross-agent memory, semantic retrieval and automatic recovery of uncertain work remain outside M6.

M6 / 02

Identity, state ownership and contracts

Three identities with different lifetimes

Identity	Lifetime and rule
Kru thread ID	Permanent for (agent_id, lane_key). Main lane uses a canonical empty lane key. Provider changes never create a new Kru thread.
Session generation	Monotonically increases within one Kru thread whenever a native session is replaced. One current generation; retain retired lineage.
Native session ID	Provider-specific address. Valid only with its provider, account/runtime scope and generation. Never use it as a permanent conversation key.

```
type KruThread = {
  id: string; agentId: string; laneKey: string;
  createdAt: string;
};

type SessionBinding = {
  threadId: string; generation: number;
  provider: string; externalSessionId: string;
  configHash: string; runtimeScopeHash: string;
};

type PreparedSession = {
  binding: SessionBinding;
  mode: "resume" | "new";
  handoffId: string | null;
};
```

Domain records versus runtime state

Event-sourced: threads; message-to-thread links; session generation/replacement facts; handoff payloads, hashes and source cutoffs; confirmed handoff use. These are durable history and rebuild deterministically from M4 events.

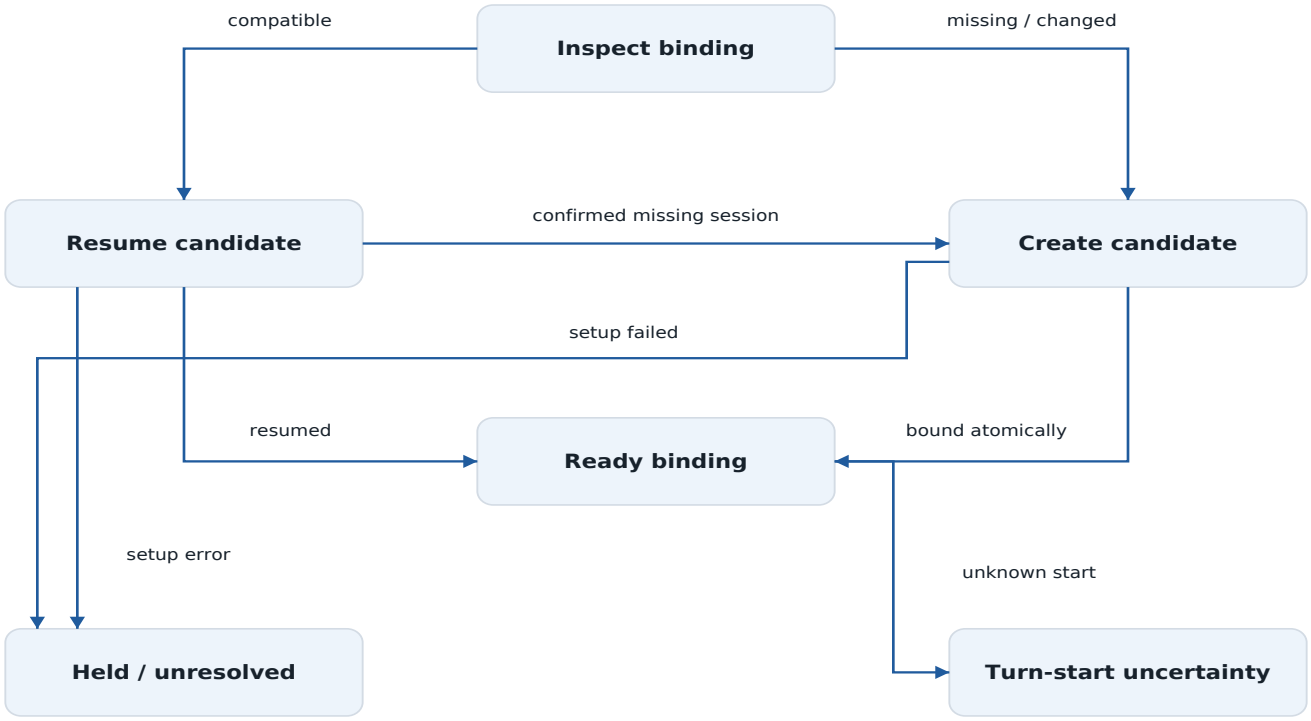
Operational: current native bindings, candidate setup attempts, process handles and readiness. Persist bindings across restart, but validate them before reuse. Replay must not start providers or recreate live process state. A rebuilt database can have no usable bindings and create a new generation from recorded history.

Respect the existing M1 port

Keep logicalThreadId explicit in engine code. M1's thread/start and thread/resume IDs remain native provider identifiers unless a deliberate port migration changes that contract. Map events using provider/runtime ownership, native thread and turn IDs to a Kru execution. M5 sender identity, reply destinations and execution IDs remain Kru-owned.

M6 / 03

Session selection and provider switching



An uncertain turn is reconciled or interrupted; never replayed automatically.

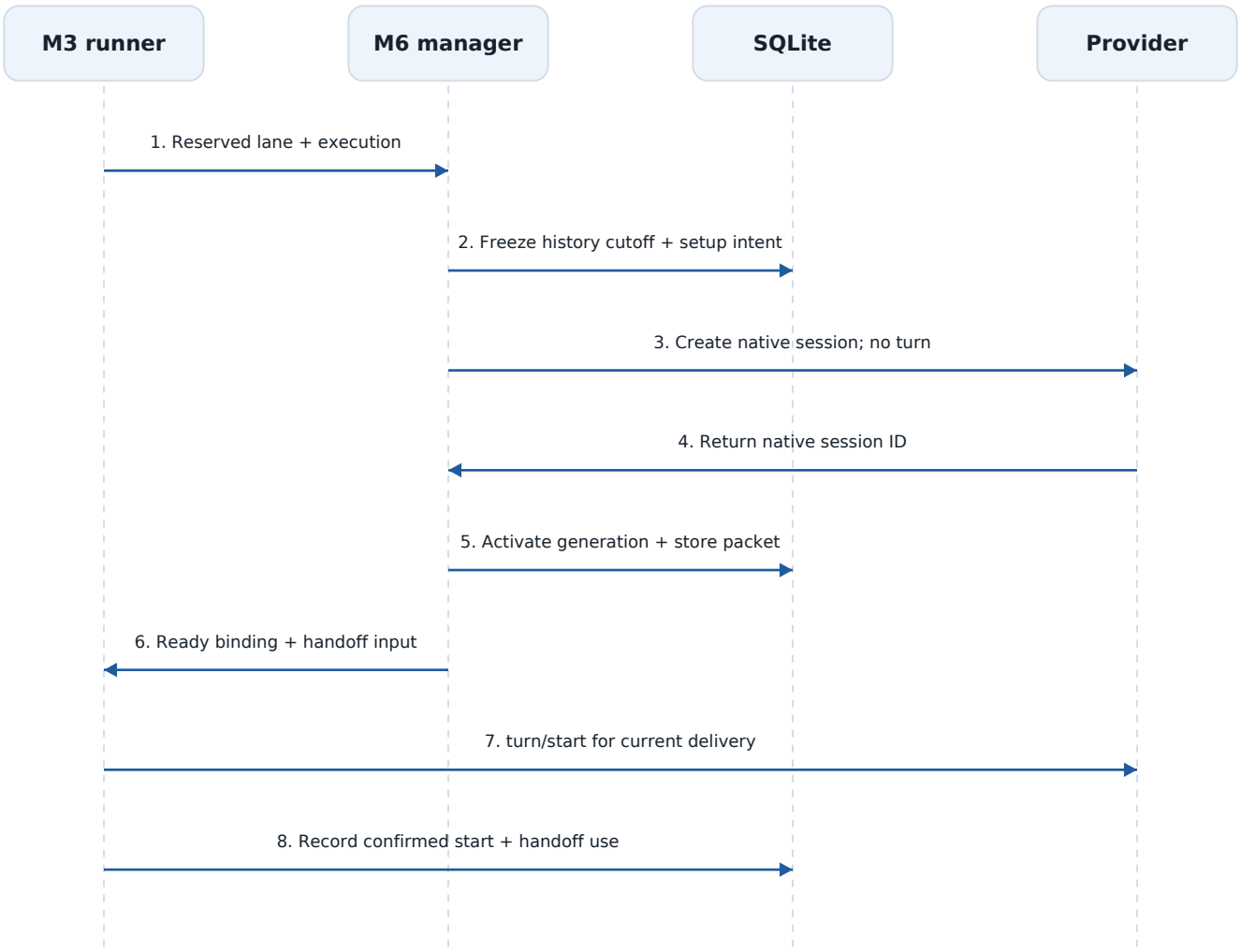
Condition	Decision
No binding; no recorded history	Create a native session. No history handoff is needed.
Compatible binding; runtime healthy	Reuse the live session, or resume after a disconnect/restart.
Confirmed missing native session	Create a replacement generation and attach a handoff.
Provider, model, prefix, tools, permissions, cwd or scope changed	Replace at the next turn boundary. Hash the complete effective session configuration.
Authentication, rate limit, transport error or setup timeout	Hold or reconcile setup. Do not pretend the native session is missing.
Switch A to B, then back to A	Create a fresh A generation from current history. Do not resume A's retired, stale conversation branch.

A configuration update during a running turn applies to the next turn. Finish the current turn under its captured configuration; do not interrupt it implicitly. Each lane transitions independently. If the requested provider cannot start, hold work visibly instead of silently using a different provider.

Use the existing lane reservation plus a compare-and-set on current generation to prevent competing activations. An already claimed execution keeps its configuration snapshot through setup; later edits take effect at the following execution.

M6 / 04

Replacement sequence and commit boundaries



Capture history at a committed event-sequence cutoff before constructing the packet. Exclude the current claimed batch: its messages are supplied once as current input. Persist the setup intent before a native create request so an interrupted setup can be inspected.

Create the candidate outside any SQLite transaction. Once creation succeeds, one dispatch records generation activation, packet metadata/payload and retirement of the old generation, and updates the operational binding. Check lane ownership, expected generation and the captured configuration before commit. If the check fails, discard or clean up the unused candidate.

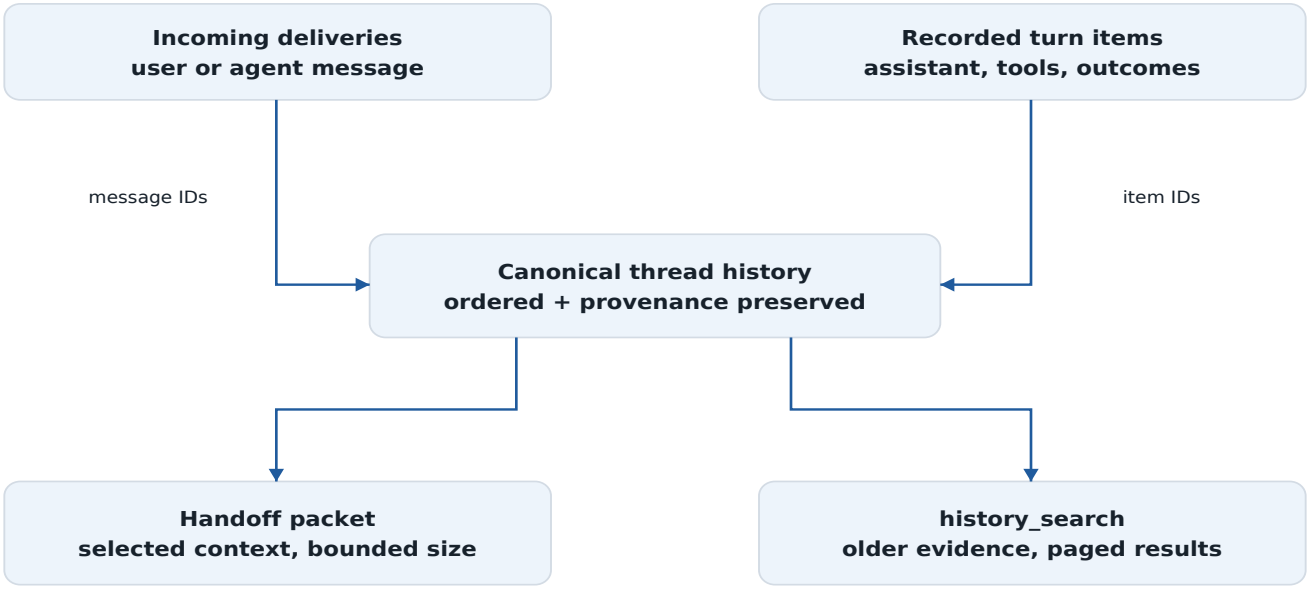
Handoff delivery is tied to the first confirmed execution

The first turn in the new generation prepends the stored packet to the current delivery input. Record handoff use against the bound execution when turn start is confirmed, not when input is merely assembled. Later turns do not repeat it. If the provider definitely never accepted input, keep the handoff pending for the next valid attempt.

If turn/start times out or the provider dies after it may have accepted input, preserve the execution and handoff association. Apply M3's unknown-start reconciliation; do not resend the prompt automatically. When the outcome remains unknown, interrupt the delivery and quarantine that native generation before allowing future work.

M6 / 05

Canonical history and bounded handoffs



The handoff is derived context. Never ingest it again as new conversation history.

Record evidence, not just assistant messages

Build the history projection from both incoming mailbox messages delivered to this thread and M4 turn items. Include roles, source message/item IDs, execution IDs, ordering and outcomes. Preserve repeated attempts as execution records without duplicating one source message as several new user requests. Associate batched messages individually with the consuming turn.

Use durable event sequence and item/message ordering, never timestamps alone. Incoming agent messages preserve their sender, reply relationship and chain. Keep failed or interrupted execution markers, completed tool results and partial-output status explicit. Do not turn incomplete text into a confirmed result.

Deterministic packet, with a fixed ceiling

Use a deterministic extractive builder in M6: no extra model call, invented summary or unmetered summarisation. Store a renderer version, source cutoff, included IDs, omissions, complete packet and hash. Proposed default: 12,000 Unicode code points, including the wrapper; also obey the runner's byte limit and any known provider input budget.

Reserve room for current delivery input first. Include thread identity and switch reason, recent task/decision excerpts, unresolved requests and recent outcomes with source IDs. Label excerpts as historical evidence and indicate truncation. Prefer whole entries, then explicit excerpts. If space is tight, keep a compact history notice and retrieval IDs; reject an oversized current input before a provider call.

Prepend the packet to the first turn's input, not the stable system prefix. Historical instructions are quoted context, not new work or tool authority. Native hidden reasoning, provider-internal plans and unrecorded context cannot be transferred. Workspaces/files remain where they are; M6 does not copy files or recreate a provider's hidden state.

M6 / 06

history_search: scoped evidence retrieval

Register history_search as M6's third Kru tool alongside M5's two tools. Update the stable tool-definition hash; existing sessions then rotate at a turn boundary. The handler reads committed history and never starts a model turn or creates a mailbox delivery.

```
type HistorySearchArgs = {
  query?: string;       // literal text search
  messageId?: string;   // exact source message/item ID
  limit?: number;      // default 6, maximum 20
  cursor?: string;     // opaque continuation
};

type HistorySearchResult = {
  threadId: string; snapshotSeq: number;
  matches: Array<{
    sourceId: string; executionId: string | null;
    role: string; seq: number; text: string;
    truncated: boolean; outcome?: string;
  }>;
  nextCursor: string | null;
  omitted: boolean;
};
```

Search contract

• For an initial call, require exactly one of query or messageId. Scope is the caller's Kru thread, derived from trusted execution context. Do not accept an arbitrary thread or agent ID. A valid ID outside the thread returns no match without revealing its owner.

• Provide case-insensitive literal text matching with deterministic newest-first ordering and sequence/ID tie breaks. Escape query syntax; do not expose arbitrary SQL or a raw FTS expression. Exact-ID retrieval can page the complete recorded body.

• Bound output by a default 8,000-character response budget and at most 20 entries. Include explicit truncation and continuation. Character counting matches the handoff builder; enforce the provider tool-result byte ceiling as well.

• A cursor fixes thread, query or source ID, snapshot sequence and last position or excerpt offset. Validate it against the caller; subsequent pages use the same committed snapshot. Do not repeat, skip or leak matches when new turns arrive.

• Return an empty successful result only when there are no matches in available recorded history. Database errors return a tool error. Known migration/history gaps are reported explicitly, not disguised as complete recall.

Retrieval preserves evidence and accounting

Return stable source IDs so the model can cite what it found. Tool calls and bounded results are recorded through M4. Do not ingest history-search results as new searchable conversation facts, or later searches will recursively return copies of old results. Retrieval adds no execution, but its tool-result tokens are part of the provider's current turn usage.

M6 / 07

Failures, migration and module boundaries

Failure or transition	Required handling
Native create times out	Outcome is setup-unknown, not session-missing. Retain setup intent; reconcile or safely abandon the candidate. No turn/start was sent.
Create succeeds; binding commit fails	No task executes. Keep enough identity for candidate cleanup; retry setup only after the old attempt is settled.
Activation committed; daemon dies before turn/start	Keep binding and packet. Boot recovery handles the claimed delivery; handoff remains pending. No automatic retry of that delivery.
Turn may have started; response lost	Follow M3 unknown-outcome policy. Quarantine uncertain session; a future fresh generation gets recorded history plus an uncertainty marker.
Old generation emits late events or tools	Fence by generation and execution. Reject new stale tool sends; preserve valid committed M5 receipt replay without new enqueue.
One agent on shared provider fails	Isolate its binding when possible; never kill all provider sessions solely to switch one agent. If a shared process dies, each affected execution is reconciled separately.

Migration and replay

Add a new migration after the actual current schema version. Create one thread per existing agent/lane and link recorded executions and incoming messages. Import current agent_sessions bindings as unverified candidates. Reuse only the most recent binding consistent with recorded execution/configuration; ambiguity forces a fresh generation, not a guessed continuation.

Write deterministic genesis events for imported thread history and session lineage. Keep native IDs in operational bindings; rebuild domain projections without resuming external sessions. Report any history that was never recorded before M4 as unavailable. Keep schema fixtures and verify migrated and new databases follow the same constraints.

Suggested ownership and build order

```
src/threads/
  types.ts             Logical thread and session contracts
  lifecycle.ts         Selection, generation fencing, activation
  history.ts           Canonical history projection queries
  handoff.ts           Bounded deterministic packet renderer
src/tools/history-search.ts
src/store/projectors/  Threads, lineage, handoff use, history
test/threads/          Lifecycle, handoff, migration, e2e tests
```

Build identity/history first, then session selection, packet rendering, history_search and runner wiring. Finish with migration/replay, fake-provider fault injection and existing adapter integration checks. Provider-native session error classification belongs in adapters; the engine depends on typed outcomes.

M6 / 08

Acceptance tests and completion standard

#	Test	Proves
1	Ten turns on A; switch to B; ask about turn one.	Same Kru thread; answer grounded in packet or retrieved source.
2	Restart with a valid native binding.	Resume without duplicate handoff or duplicated input.
3	Delete the native cache; resume reports missing.	Fresh generation and bounded handoff preserve recorded context.
4	Return auth/rate-limit/transient errors on resume.	Visible hold; no false missing-session replacement.
5	Switch A to B to A after new work on B.	Fresh A generation; no stale branch reuse.
6	Change provider/config while a turn runs.	Current snapshot finishes; next execution uses new config.
7	Race setup/activation and emit late old events.	One current generation; stale events cannot mutate new execution.
8	Timeout create; fail activation; crash before start.	Setup intent, candidate cleanup and pending handoff are consistent.
9	Lose turn/start response after input acceptance.	No automatic resend; uncertain generation is quarantined.
10	Use large Unicode history and a large current input.	Hard character/byte bounds; current input appears once.
11	Resume several turns after successful handoff.	Packet attached once per generation; stable prefix stays stable.
12	Search first-turn evidence omitted from handoff.	Exact IDs and paged excerpts retrieve committed original content.
13	Search from another agent/lane; tamper with cursor.	No cross-thread disclosure; invalid cursor rejected.
14	Append history between search pages; simulate DB error.	Stable pagination; failure distinct from no matches.
15	Migrate prior DB; replay twice into empty DBs.	Deterministic history/lineage; no provider side effects.
16	Carry M5 request/reply through provider replacement.	Origin lane, execution IDs, chain and receipts remain intact.
17	Count provider turn starts and usage records.	No hidden summary turn; each turn has delivery and ledger entry.
18	Run missing-session/switch cases through each adapter.	Port parity; provider-specific errors classified correctly.

Done when: all 18 fake-backed cases pass in ten repeated runs; strict type checks, migration/replay and relevant existing suites pass. The ten-turn switch demo answers using recorded evidence, including facts outside the handoff. Use opt-in real-adapter checks for native resume, replacement and tool registration.

Read after: ensureThread in src/backend/agent/thread-lifecycle.ts; src/backend/agent/handoff-history.ts; History retrieval in docs/architecture/providers.md. Follow the project's independent implementation rule.

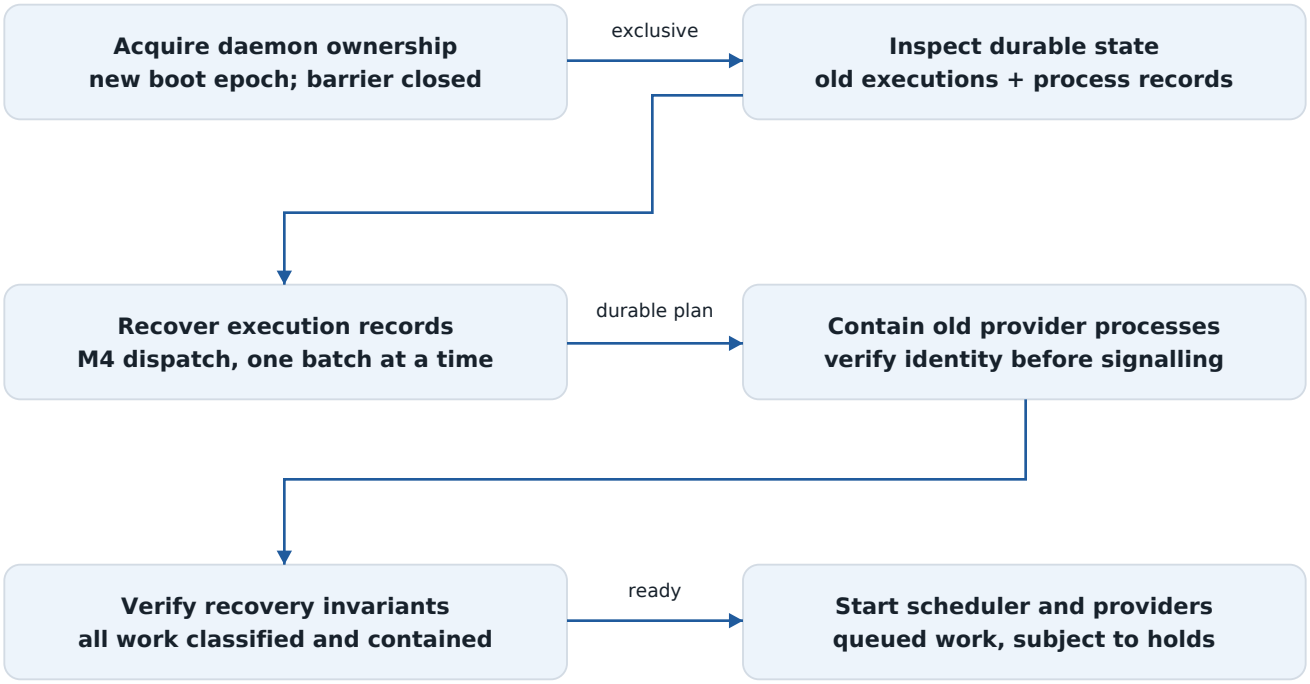
You can now: use kru agent set <agent> --provider codex while retaining the Kru thread, recorded history and outstanding mailbox work. Continuity means access to recorded evidence, not a guarantee that all native provider context or model behaviour is identical. M7 remains the crash-recovery milestone for v0.2.

M7 / 01

Crash recovery and safe restart

Status: planned implementation specification. Expanded 4 November 2026. M7 completes the v0.2 recovery architecture; it is not a claim that recovery has already been implemented or verified.

Purpose. After an abrupt daemon exit, preserve accepted work, classify abandoned executions honestly, contain old provider processes and resume only queued work that passes normal holds. Unknown outcomes become interrupted and require an explicit user decision before retry.



Any unresolved ownership or containment issue keeps the boot barrier closed.

The boot barrier is the central control

No new provider runtime starts, no session resumes and no delivery is claimed until recovery has reconciled the previous daemon's work and contained its owned processes. Read-only inspection and new mailbox enqueues can remain available; those new messages stay queued while the barrier is closed.

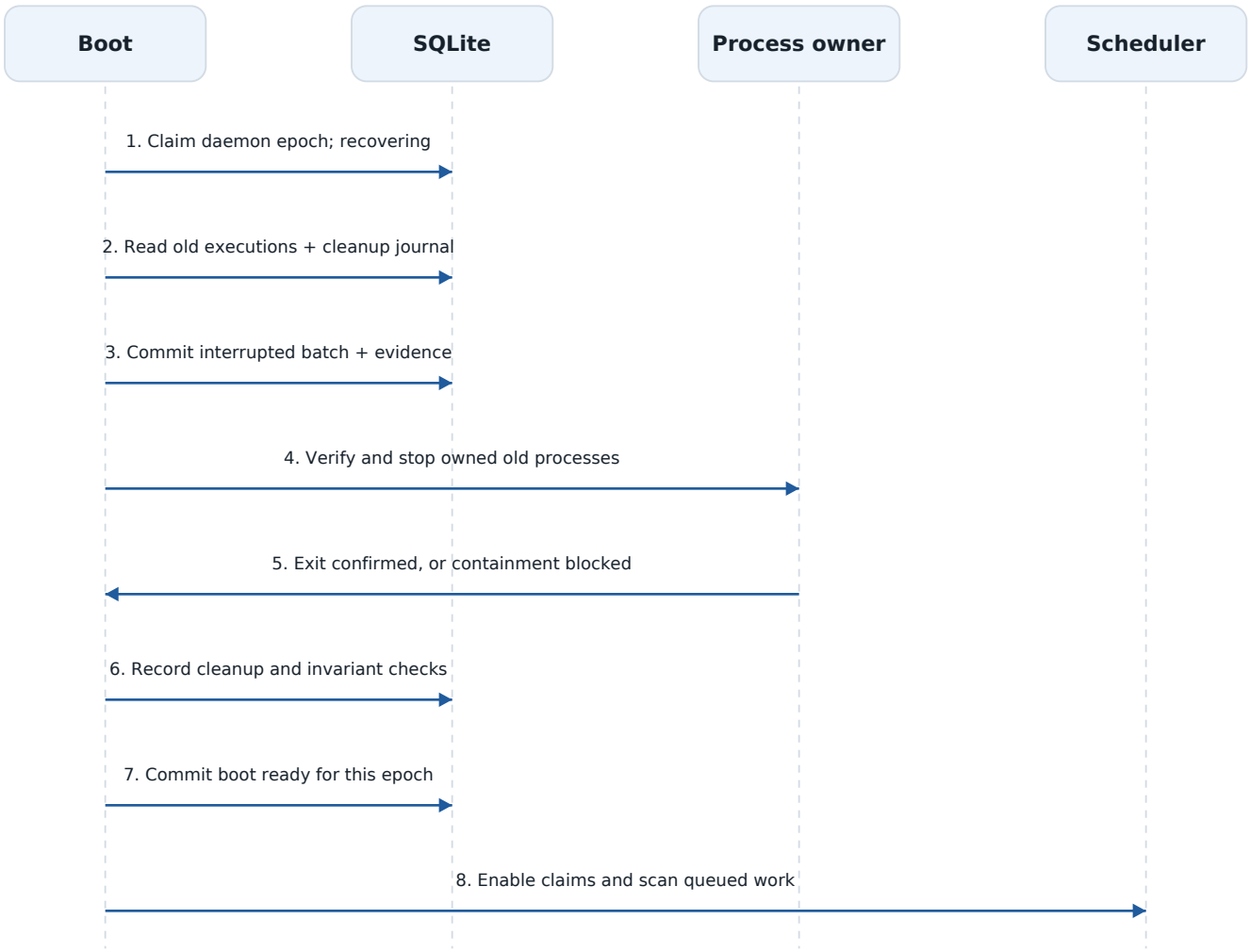
What M7 guarantees

Committed messages and receipts remain durable; terminal states never reopen; one claimed batch is recovered atomically; stale events cannot mutate a new execution. Kru records uncertainty rather than claiming that an external action did not happen.

File writes, shell commands and external API effects cannot be rolled back or made exactly-once by a mailbox recovery transaction. M7 does not automatically replay unknown turns, repair a corrupted database, restore a backup or transfer live execution between machines.

M7 / 02

Boot phases and daemon ownership



Exclusive owner first. Open the database with existing WAL, foreign-key and busy-timeout settings; perform the required schema checks/migrations through the existing exclusive path. Atomically acquire the daemon lock before recovery writes. A second live matching daemon is refused, even if its heartbeat looks old.

Record a boot epoch, PID and process start identity scoped to this database and host. A stale PID is not enough to prove ownership: compare the current process identity. A missing old owner permits takeover; unverifiable ownership blocks startup with a diagnostic. Heartbeats describe liveness but do not authorise stealing a live owner's lock.

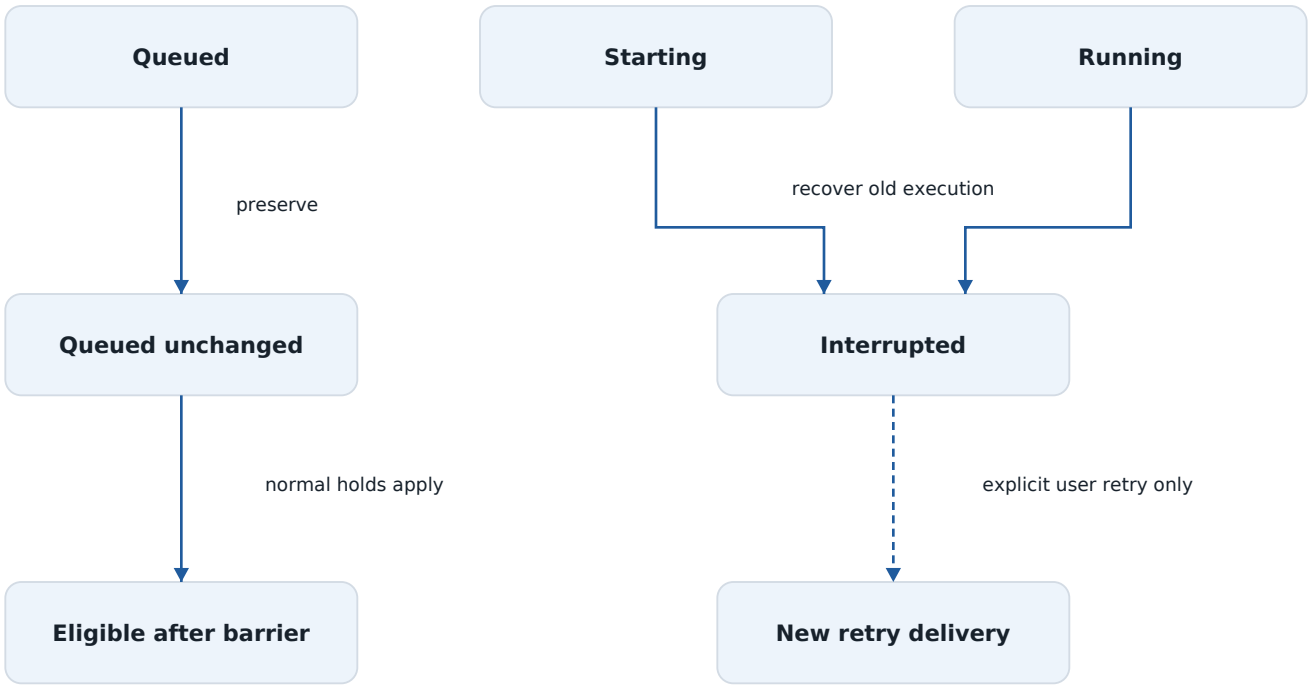
Recovery progresses through durable phases

inspect → reconcile → contain → verify → ready. Store progress for diagnosis, but derive safety from current records on every restart. Recheck ownership and epoch on recovery writes and before enabling scheduling. Losing ownership immediately disables claims and provider requests.

Process operations run outside SQLite transactions. Every phase is restartable: a second crash repeats checks against committed facts. Only an atomic ready transition for the current epoch opens the barrier. After that, the scheduler performs its ordinary startup scan and budget/slot/paused/provider checks.

M7 / 03

Delivery, lane and session state transitions



Completed / failed / interrupted / cancelled deliveries remain terminal.

Recovered record	Required outcome
Old starting or running batch	Mark every member interrupted in one command, with a daemon-restart/unknown-outcome reason. Do not restore it to queued at boot.
Queued delivery	Preserve message, sequence, lane, chain, hop and receipt. Run only after the boot barrier and ordinary holds clear.
Terminal delivery or completed item	Preserve its recorded outcome and content. Never infer a different result from a stale process message.
Active lane reservation	Release as part of recovery of its batch. An orphan reservation requires an audited repair or a blocked invariant check, not silent deletion.
Generation tied to uncertain execution	Quarantine for execution. Future work uses an M6 replacement and uncertainty-aware handoff; never resume the ambiguous turn.
Idle compatible session binding	Retain as unverified across restart. After containment and boot readiness, M6 may resume its native session cache.
Setup intent with no turn sent	Reconcile candidate ownership/cleanup. Keep an unused committed handoff pending; the abandoned starting delivery still ends interrupted.

This conservative boot rule applies even when starting may mean that setup had not finished. Live-process M3 reconciliation can use evidence to settle a start; M7 never fabricates successful completion from provider cache contents. Clear stale runtime handles, not durable threads or conversation history.

M7 / 04

Atomic recovery commands and durable evidence

Recover one claimed execution at a time

Reuse the M3 batch/execution identity. A recovery command checks the current daemon epoch and the expected old execution state, then uses M4 dispatch to commit all related domain changes and operational cleanup. Never run process termination inside the transaction.

```
recoverExecution({
  executionId,
  expectedBatchId,
  ownerEpoch,
  reason: "daemon_restarted_outcome_unknown",
  recoveredAt,
  outputCheckpoints,
  usageCheckpoint,
});

commandKey = hash([
  "kru.recover-execution.v1", executionId
]);
```

The command key is stable across repeated boots. Store its result through M4 command receipts and also enforce terminal-state checks. A replay returns the prior result without appending duplicate interruptions, partial items or budget debits. IDs and timestamps are command inputs or event payload values, never generated by replay projectors.

In the same dispatch	Why
Delivery terminal transitions for the complete batch	No partially recovered batch or member left running.
Execution/turn interruption evidence	Associate the interruption with Kru's execution. A native turn ID may be absent; do not invent one.
Partial-output and usage-gap records	Preserve available evidence without pretending it is a completed response or final usage.
Session quarantine and lane release	A freed lane cannot accidentally reuse an uncertain native generation.
Cleanup journal links and stored command receipt	Retain old process ownership evidence even after active lane rows disappear.

M4 remains the only transaction owner

If recovery fails before commit, none of the batch changes are visible. If it commits and the daemon dies again, the next boot sees interrupted work and pending cleanup. The barrier stays closed until containment completes. A turn that already committed its ordinary M4 final transaction is skipped.

Use bounded transactions per execution, not one unbounded transaction for the entire history. Take a recovery cutoff/old epoch so concurrent CLI enqueues are never swept into abandoned work. If recorded rows disagree with replayable events, stop and report the invariant violation; do not guess missing outcomes.

M7 / 05

Process containment and stale-event fencing

A released lane does not prove its old process stopped

Use M0's persisted process ownership records and M3's daemon identity. The recovery journal must retain the database/runtime identity, owning boot, PID, process start identity and process-tree/group evidence needed by the supported operating system. Treat these records as ownership evidence, not as permission to signal any matching PID.

Observed condition	Action
Verified owned old runtime is alive	Use the established graceful-stop deadline, then force-stop its verified owned process tree if necessary. Confirm exit before opening the barrier.
Recorded runtime is already gone	Mark cleanup complete; retain audit facts. A later reused PID must not be signalled.
PID now belongs to a different start identity	Do not signal it. Resolve the old process-tree evidence separately; PID reuse is not proof that all original descendants are gone.
Group leader exited; descendants may remain	Use supported verifiable tree/group ownership evidence. Never kill a process group based on its numeric ID alone.
Invalid/truncated record, permission denial or unverified descendants	Keep recovery blocked if containment cannot be established. Report the precise record/runtime; do not silently discard uncertainty.
Shared provider runtime from the dead daemon	Clean it up once, then reconcile all linked executions. Idle session files can still be resumed later; the old runtime itself is not adopted.

No database lock while waiting for the operating system

Persist the cleanup intent, perform bounded process actions outside dispatch, then record the observed result. If a crash occurs after a kill but before recording success, repeat the identity check: a gone process is already contained. Never assume a timeout means termination succeeded.

Fence every new side effect

At provider-event and tool-call boundaries, resolve the tuple of daemon epoch, runtime identity, session generation and execution. A stale completion cannot finish a new batch or release its lane. A stale new send_message is refused; an authenticated replay of a committed M5 invocation may return its existing receipt without new writes.

Detached descendants or external actions outside Kru's verifiable ownership remain a reported limitation. M7 does not introduce a sandbox. Validate process-tree behaviour on each supported platform; unsupported containment must be visible rather than assumed.

M7 / 06

Interrupted output, usage gaps and budgets

Keep partial evidence explicitly partial

M4 streams deltas from memory and may save periodic partial snapshots. Recovery can preserve only the latest committed checkpoint: any later in-memory text may be lost. Do not promise to recover every token. Existing completed items remain complete; only unfinished items become interrupted partial records.

Store a versioned recovery event with execution/item identity, checkpoint sequence, text and an interrupted/partial marker. Preserve source IDs and avoid creating a second user message. M6 history and handoff rendering must show the uncertainty. Never turn partial text into a successful final response.

Final usage and observed usage are different facts

Available evidence	Ledger and reporting
Ordinary final transaction already committed	Keep its final usage and stored price exactly as recorded. Do not add a recovery charge.
Latest durable usage snapshot exists	Record the observed counters once, marked partial. Values are snapshots, not a sum of repeated notifications; usage after the checkpoint remains unknown.
No durable usage snapshot	Record a missing-usage gap for the execution. Do not invent zero tokens, a free turn or a final cost.
Observed tokens but no price	Preserve NULL cost and unpriced status, independently of whether the observation is partial or final.

Add a durable latest-usage checkpoint if the current implementation only keeps in-flight counts in memory. Coalesce writes; checkpoint latency defines the evidence gap. Before clearing a checkpoint, promote needed evidence through dispatch so recovery projections remain replayable.

Keep final usage_records semantics intact. Add a versioned recovery-usage projection/gap record keyed by execution instead of inserting fabricated final usage. Reports separate final cost, observed partial cost, unpriced entries and missing-usage executions. Each execution contributes once to observed totals; completeness and estimate flags are separate.

Budget gaps cannot silently reset spending

Reload committed usage and observed partial usage at startup using original accounting timestamps and M4 timezone rules; do not move historical spend to the reboot date. Preserve M5 chain attribution. When unknown usage prevents evaluating a configured finite budget, hold the affected agent until an explicit operator resolution or conservative recorded reserve is supplied. Releasing that hold is audited and does not claim exact cost or retry the work.

M7 / 07

Operator workflow, storage and module boundaries

Make the recovery result inspectable

Extend kru status and its JSON form with the boot phase, barrier state, interrupted execution/delivery IDs, quarantined sessions, cleanup blockers and usage gaps. The final recovery report distinguishes preserved queued work from interrupted attempts and states why an affected agent remains held.

Proposed inspection command: kru recovery inspect, with --json for automation. It reads the durable report and does not start providers. Operational resolution is explicit and validated; an acknowledgement alone cannot bypass an unresolved process-containment check.

Manual retry is a new attempt

The user inspects the recorded output and any external effects, then uses the existing kru retry <delivery>. M2 creates a new delivery with retry_of; the old one stays interrupted. Preserve M5 chain/hop and charge the new attempt against its applicable chain limit. M6 selects a safe generation and renders an uncertainty-aware handoff.

A retry can repeat a file edit or external action that the first attempt already performed. No boot routine decides to retry on the user's behalf. Resolving a usage hold or process cleanup issue does not implicitly enqueue a retry.

Storage responsibilities

State	Storage rule
Recovery decisions, interrupted items, usage gaps, session quarantine	Versioned M4 events and deterministic projections. Retain evidence for inspection and replay.
Boot phase/epoch, cleanup intents, process observations, checkpoints	Durable operational state. Revalidate on restart; never recreate processes during event replay.
M5 messages, replies and receipts	Preserve every committed send even when its sender is interrupted. A rolled-back send is absent; do not synthesise it.
M6 thread history and idle bindings	Keep logical history. Revalidate bindings; isolate uncertain generations rather than wiping all native caches.

```
src/boot/
  recovery.ts          Boot barrier and phase orchestration
  inspect.ts           Old execution and ownership inventory
  recover-execution.ts Atomic M4 recovery command
  containment.ts       M0 ownership checks and process cleanup
  report.ts            Durable recovery report and status view
src/store/projectors/  Recovery evidence and usage-gap views
test/boot/             Crash checkpoints and restart harness
```

Add a migration after the actual current schema version, with frozen fixtures and replay checks. Do not modify shipped event shapes. Reuse M0 ownership and M3 lock/scheduler code; do not create a competing lock, mailbox or transaction manager.

M7 / 08

Crash-injection tests: core recovery

Use child-process fake providers with named crash checkpoints. Tests synchronise on observed events or pipes before sending SIGKILL, not arbitrary sleeps. Restart against the same on-disk database and inspect both stored state and live owned processes.

#	Scenario	Required result
1	Kill daemon while a fake turn is running.	Old delivery interrupted; queued successor runs only after containment.
2	Kill after claim but before turn/start is sent.	Starting batch interrupted conservatively; no automatic restore or retry.
3	Kill after provider accepted start but before acknowledgement.	Unknown execution is not resent; generation quarantined.
4	Kill after provider completes but before M4 final commit.	No invented success; attempt interrupted with available evidence.
5	Kill immediately after ordinary final commit.	Completed state/output/usage remain; no extra recovery charge.
6	Inject failure halfway through batch recovery dispatch.	All batch changes roll back; no partial terminal transitions or lane release.
7	Kill after recovery commit but before process cleanup.	Next boot resumes cleanup without duplicate recovery events.
8	Kill after process exit but before cleanup result commit.	Next boot observes gone process; no signal to a reused PID.
9	Repeat recovery over an already recovered database.	Same receipts/results; no duplicate partial items, gaps or cost.
10	Start two daemons against one database.	Only one owner recovers or schedules work.
11	Pause a live owner until its heartbeat looks stale.	Second daemon refuses takeover; heartbeat age is not lock authority.
12	Reuse recorded PID with a different start identity.	Unrelated process is untouched; tree containment is separately verified.

Observe the invariants, not just exit codes

After a successful boot: no old execution remains starting/running; no stale lane reservation blocks queued work; uncertain sessions are quarantined; the new epoch owns all claims; the process journal has no unresolved containment blocker. Terminal rows, original queue order, receipts and thread history are preserved.

A deliberately blocked boot must show the exact blocker and start zero providers or turns. A safe failure to start is an expected test result when process ownership or database state cannot be established.

M7 / 09

Integration tests and v0.2 completion

#	Scenario	Required result
13	Old owned runtime and child outlive the dead daemon.	Verified tree is stopped; scheduler stays gated until containment.
14	Permission denial, malformed ownership or orphan descendants.	Clear blocked report; no unsafe process signal or startup bypass.
15	Shared runtime contains active and idle agent sessions.	One runtime cleanup; active attempts interrupted, idle bindings retained for validation.
16	Enqueue from another CLI while boot recovery runs.	New work stays queued with its sequence; it is never swept as abandoned.
17	M5 send commits; sender crashes before receiving receipt.	Recipient work and reply links survive; same-call replay adds nothing.
18	Crash with partial text and repeated usage snapshots.	Latest durable partial evidence only; no duplicate tokens or invented final answer.
19	Restart with partial/missing usage and a finite budget.	No zero-cost assumption; explicit gap and applicable budget hold.
20	Late old-generation events/tools arrive after restart.	No new send, completion or lane release against the new epoch.
21	M6 setup intent or pending handoff exists during crash.	Candidate reconciled; packet preserved; no automatic delivery retry.
22	User retries an interrupted agent delivery.	New retry_of attempt; chain cap honoured; old terminal row unchanged.
23	Migration/replay from prior fixture; injected DB inconsistency.	Deterministic evidence; inconsistent state blocks rather than being guessed away.
24	Graceful shutdown followed by forced second signal.	First path settles known work; forced path recovers safely on next boot.

Implementation order

1. Add daemon epoch and explicit boot barrier around existing startup.
2. Inventory old batches, session generations and owned processes.
3. Implement atomic recoverExecution and replayable evidence.
4. Wire verified process containment and restartable cleanup journal.
5. Add usage-gap accounting, status/reporting and recovery holds.
6. Run crash-injection, migration and platform-specific process tests.

Done when: all 24 deterministic cases pass in ten repeated runs; strict type checks and relevant M0-M6 regression suites pass. Run the real SIGKILL/restart test and owned-child cleanup on each supported platform. Optional live-adapter smoke tests verify session handling without replacing fake fault coverage.

Read after: src/backend/agent/boot-recovery.ts, plus the project's M0 process-ownership, M3 daemon-lock and M4 dispatch implementations. Follow the independent implementation rule for OpenBot references.

You can now: run Kru as a service with durable queues, explicit interruption records and controlled restart behaviour. Publish the limits and manual-retry procedure, complete release checks and release v0.2. The HTTP/SSE API and later product features remain on the road to v1.0.

Road to v1.0

After M7 the engine is complete. v1.0 adds what makes Kru useful to people who never read its code: an API, scheduled work, and limits that keep it safe on a shared machine.

Feature	What it adds	Read in OpenBot
HTTP + SSE API	Send, list, stream and cancel over HTTP, so any tool can drive Kru	`src/main/team-api-server.ts` (shape only)
Routines	A timer or webhook that enqueues a delivery; no new execution path	`src/backend/agent/routine-scheduler.ts`, `docs/architecture/events.md`
Context compaction	Ask the provider to compact near 80% of the window; rotate the session when it cannot	`src/backend/agent/context-compaction.ts`
Global turn slots	A cap on concurrent turns, people's messages first	`src/backend/agent/turn-slots.ts`
Deferred tool discovery	`tool_search` and `tool_call` keep the always-loaded tool list small	`src/backend/agent/tool-catalog.ts`
Agent memory	`remember`, `search_memories`, `forget_memory` tools on a SQLite table	`src/backend/agent/agent-memories.ts`
Workspace sandbox	Each agent writes only to its own folder plus allowed roots	`src/backend/agent/workspace-sandbox.ts`
Usage limits	Hold a lane when the provider reports a limit; resume when it resets	`src/backend/agent/usage-limit-gate.ts`

Build the API first. It turns Kru from a CLI into a platform, and every later feature can be tested through it.

Release checklist

People adopt a tool they can install and see working in five minutes. Do these before you announce v0.1.

- [ ] Check that the name "kru" is free on npm and GitHub; short names are often taken, so a scoped package such as @<your-name>/kru is the fallback
- [ ] Add an MIT `LICENSE` and a note that the project is inspired by OpenBot and shares no code with it
- [ ] One-line install: `bunx kru` or `npm i -g kru`
- [ ] README with a 30-second demo GIF: two agents hand work to each other
- [ ] A quick start that works with the fake provider, so a reader needs no API key to try it
- [ ] `examples/`: coder + reviewer pair, a Slack bot over the HTTP API, a nightly routine
- [ ] `docs/architecture.md` with the lifecycle diagram from this doc
- [ ] CI that runs every test against the fake providers
- [ ] A security note: agents run with the provider's own permissions; Kru adds no sandbox until v1.0
- [ ] Secrets are redacted from logs and stored error messages
- [ ] A `CHANGELOG.md` and semantic version tags
- [ ] A write-up of what you learned at each milestone, for your portfolio

How to read the OpenBot source

Read the docs for *why*, the tests for *what*, and the history for *how they got there*. File paths in this doc are from nightly-labs/openbot at commit `0b311bf` (v0.57.0).

1. Clone the full history, not a shallow copy: `git clone https://github.com/nightly-labs/openbot.git`.
2. Read `docs/architecture/structure.md`, `providers.md` and `src/backend/AGENTS.md` before any code. They state the rules the code obeys.
3. Read `docs/glossary.md`. It maps product words (thread, side thread, delivery, lane) to code names.
4. Treat `scripts/*-e2e.ts` and `scripts/provider-fakes/agents/*.ts` as behaviour specs. `stall.ts` and `session-errors.ts` show the failures they guard against.
5. Use `git log -p -- <file>` on the file of your current milestone. Bug fixes show the edge cases you will hit.

6. Follow one message by searching method names: `enqueue` → `scheduleDrain` → `startDelivery` → `"turn/start"` → `handleNotification` → `completeTurn`.
7. Skip `src/renderer`, `src/main` and `apps/` at first. They are product surface, not engine.

Sources

- nightly-labs/openbot on GitHub — repository read at commit `0b311bf`, 2026-11-01